
InferTutor Arena

Capstone Project Documentation

Production Multimodal LLM Serving on Modal with vLLM

Author:	Manoj Chandrashekar
Date:	June 2026
Model:	Qwen3-VL-4B-Instruct
Infrastructure:	Modal (H100 GPUs) + vLLM 0.21.0
Best Score:	219,235,337 (Text Track)
Mixed Score:	6,369,520 (Mixed Track)

“Deploy, load-test, measure, and optimize a real multimodal LLM serving pipeline under concurrent production traffic — then engineer it to beat the reference baseline on every metric.”

Contents

1	Introduction	3
1.1	Project Overview	3
1.2	The Product Story	3
1.3	Competition Tracks	3
2	System Architecture	4
2.1	Infrastructure Stack	4
2.1.1	Modal (Serverless GPU Compute)	4
2.1.2	vLLM 0.21.0	5
2.1.3	Qwen/Qwen3-VL-4B-Instruct	5
2.2	Scoring Formula	5
2.3	Workload Specification	6
3	Environment Setup	6
3.1	Prerequisites and Installation	6
3.2	Modal Authentication	6
3.3	Hugging Face Token and Model Access	7
3.4	Smoke Test	7
3.5	How the Runner Works	8
4	Text Speed Track — Experiments	9
4.1	Experiment Design Philosophy	9
4.2	Experiment 1: Baseline (50 Users, 1 GPU)	9
4.2.1	Baseline Analysis	10
4.3	Experiment 2: Saturation Test (100 Users, 1 GPU)	11
4.4	Experiment 3: Prefix Cache OFF (50 Users, 1 GPU)	12
4.5	Experiment 4: Chunked Prefill OFF (50 Users, 1 GPU)	13
4.6	Experiment 5: Scale to 2 Replicas (100 Users)	14
4.7	Experiment 6: Scale to 4 Replicas (200 Users)	16
4.8	Experiment 7: Compiled Mode Breakthrough (4 Replicas, 400 Users)	17
4.9	Experiments 8–9: Pushing the Ceiling (500 and 600 Users)	19
4.10	Experiment 10: Boss Fight (8 Replicas, 600 Users)	21
5	Multimodal Product Track — Experiments	23
5.1	Experiment Design for Multimodal Traffic	23
5.2	Experiment 11: Mixed Baseline with Learned Knobs (120 Users, 4 GPUs)	23
5.3	Experiment 12: Chunked Prefill OFF (Mixed, 120 Users)	25
5.4	Experiment 13: User Count Ceiling (200 Users, Mixed)	26
5.5	Experiment 14: Image Pixel Budget (200 Users)	27
5.6	Experiment 15: Pixel Budget at Healthy Load (120 Users)	29
5.7	Experiment 16: Confirming the 120-User Ceiling (150 Users)	31
6	Complete Results Summary	32
6.1	Full Experiment Table	33

6.2	Score Progression Charts	34
6.3	Configuration Comparison	36
7	Key Findings and Lessons Learned	36
7.1	Finding 1: CUDA Graphs Transformed Text Track ITL	36
7.2	Finding 2: Prefix Caching Overhead Exceeds Savings at Short Prefixes . .	37
7.3	Finding 3: Load Balancer Behavior at 2 vs 4 Replicas	38
7.4	Finding 4: Chunked Prefill Protects Mixed-Mode TTFT	39
7.5	Finding 5: Context-Dependent Knob Effects (Pixel Budget)	40
7.6	Finding 6: Ramp-Up Strategy Affects Error Rate for Large-Scale Runs . .	41
8	Final Configurations	41
8.1	Text Speed Track — Final Configuration	41
8.2	Multimodal Product Track — Final Configuration	42
9	vLLM Architecture Deep Dive	43
9.1	PagedAttention	43
9.2	Continuous Batching	44
9.3	Chunked Prefill	44
9.4	CUDA Graph Compilation	44
10	What We Would Try Next	45
11	Conclusion	45

1. Introduction

1.1. Project Overview

InferTutor Arena is a capstone project in production LLM serving. The challenge: design, deploy, and optimize a production-grade multimodal Large Language Model serving system capable of handling high-concurrency traffic, then demonstrate engineering prowess by systematically improving every measurable production metric.

Unlike toy examples, this project uses real GPU compute (NVIDIA H100s on Modal's serverless platform), a real production inference engine (vLLM 0.21.0), and a real scoring formula that punishes inefficiency in every dimension simultaneously.

Note

The core challenge: Maximize the competition score by jointly optimizing Time-to-First-Token (TTFT), Inter-Token Latency (ITL), aggregate throughput, sustained user count, and error rate — all at the same time, while staying within a GPU budget. Every configuration decision involves trade-offs between these metrics.

1.2. The Product Story

The scenario frames InferTutor as an AI tutoring service built on top of a multimodal LLM. Users submit questions — some are short text queries, some are long reading comprehension passages, and some include images (diagrams, equations, charts). The model generates streamed responses. The operator cares about:

- **Time to First Token (TTFT):** How quickly does the first word appear? Students abandon sessions if the wait exceeds ~ 2 seconds.
- **Inter-Token Latency (ITL):** How smoothly does the response stream? Jerky output feels broken even if total time is acceptable.
- **Throughput:** How many students can the system serve simultaneously?
- **Error Rate:** Dropped requests are students left without help.
- **GPU Cost:** Every extra H100 costs money. An 8-GPU system needs to earn its overhead.

1.3. Competition Tracks

Track	Mode	Max GPUs	Traffic Mix
Multimodal Product Track (Main)	<code>-mode mixed</code>	4 $\times$ H100	55% text, 20% long, 25% image
Text Speed Track	<code>-mode text</code>	4 $\times$ H100	100% text
Boss Fight (Optional)	Either	8 $\times$ H100	Any

Note

The Multimodal Product Track is the **primary capstone track**. It requires deploying and optimizing a serving system that handles heterogeneous traffic — a more realistic production scenario than pure-text serving.

2. System Architecture

2.1. Infrastructure Stack

The serving stack consists of three layers working together:

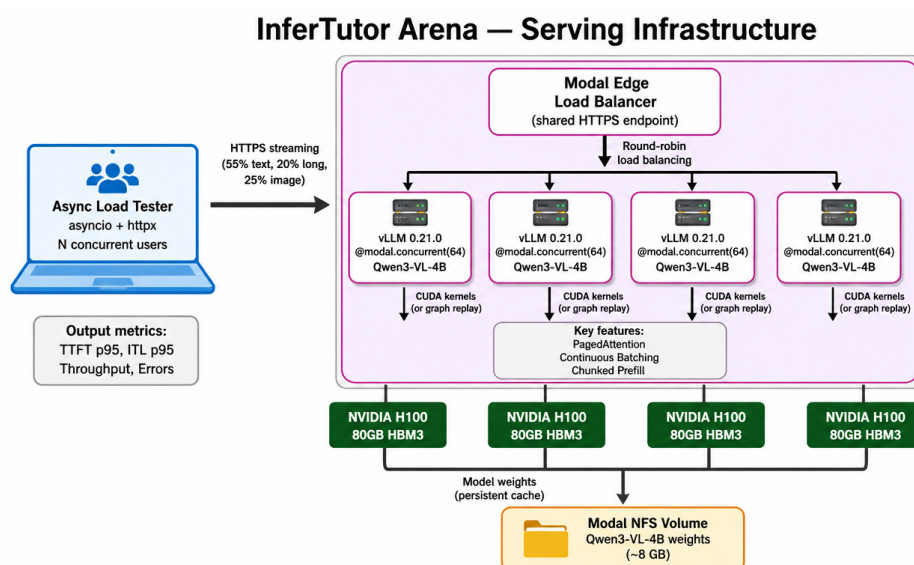


Figure 1: InferTutor Arena system architecture: load tester → Modal load balancer → vLLM containers → H100 GPUs

2.1.1. Modal (Serverless GPU Compute)

Modal provides containerized GPU execution with HTTPS endpoints. Each *replica* (container) runs one vLLM instance backed by one H100 GPU. Modal handles:

- Container lifecycle (cold start, scale-up, scale-down)
- HTTPS endpoint with a shared load-balancing URL across all replicas
- Persistent NFS volumes for model weight caching (avoids re-downloading Qwen3-VL weights on each deploy)
- `@modal.concurrent(max_inputs=N)` decorator to control concurrency per container

2.1.2. vLLM 0.21.0

vLLM is the inference engine. It exposes an OpenAI-compatible API and implements several production optimizations:

- **PagedAttention:** Manages KV cache using virtual memory blocks, eliminating fragmentation and enabling higher batch occupancy.
- **Continuous Batching:** Inserts new requests mid-generation so the GPU is never idle waiting for a long sequence to finish.
- **Chunked Prefill:** Breaks large prefills into scheduler-interleaved chunks so image/long-context requests don't block short text requests.
- **Prefix Caching:** Reuses KV blocks for shared prompt prefixes (system prompts shared across requests).
- **CUDA Graph Compilation (Compiled Mode):** Pre-captures the decode loop as a CUDA execution graph, eliminating per-step kernel launch overhead.

2.1.3. Qwen/Qwen3-VL-4B-Instruct

The model is Qwen's 4-billion-parameter Vision-Language Model:

- Supports text and image inputs simultaneously
- Uses a dynamic resolution image processor: images are tiled into $(H/28) \times (W/28)$ patches, so pixel count directly controls compute
- Runs in bfloat16 on H100
- Fits comfortably within H100's 80 GB VRAM with room for a large KV cache

2.2. Scoring Formula

The competition score is computed as:

$$\text{Score} = \frac{\text{goodput} \times \text{users} \times (1 - \text{error_rate})}{p95_TTFT_{[s]} \times p95_ITL_{[s]} \times \text{GPU_count}} \quad (1)$$

where $\text{goodput} = \text{aggregate_chunks/s}$.

Note

What the formula rewards:

- **High throughput** (chunks/s in the numerator)
- **High user count** — a system serving 400 users at the same latency beats one serving 100 users
- **Zero errors** — every dropped request is subtracted from goodput
- **Low TTFT and ITL** — both are in the denominator; halving either doubles the score
- **GPU efficiency** — using more GPUs must be justified by proportional gains

Warning

The scoring trap: Adding GPUs divides the score by GPU count. An 8-GPU system must deliver more than $2\times$ the score of 4 GPUs just to break even. If errors increase significantly (as they did in our 8-GPU boss fight), more GPUs can actually *lower* the score.

2.3. Workload Specification

The load tester (`load_test_infertutor.py`) uses a fixed `prompts.json` file with:

- 1 system prompt (≈ 40 tokens) shared by all requests
- 10 text prompts, 2 long-context prompts, 4 image prompts
- Mixed mode samples: 55% text, 20% long, 25% image
- Each request streams up to `max_tokens=96` tokens

The tester uses `asyncio` with `httpx` streaming, measures TTFT as the time from request submission to first content chunk, and ITL as the mean gap between consecutive content chunks.

3. Environment Setup

3.1. Prerequisites and Installation

```
cd starter_code
pip install -r requirements.txt
```

Listing 1: Install dependencies and change to project directory

The `requirements.txt` includes:

- `modal>=1.4.0` — Modal SDK for deployment and management
- `httpx>=0.28.0` — async HTTP client for streaming load tests
- `rich>=13.9.0` — terminal formatting for result display
- `fpdf2>=2.8.0` — PDF report generation

3.2. Modal Authentication

Modal requires a free account and an API token to deploy GPU workloads.

```
modal token new
```

Listing 2: Authenticate with Modal

This opens a browser window to authenticate. After completing, the token is stored in `~/.modal.toml`.

```
modal token info
Token: [REDACTED]
Workspace: mcrao ([REDACTED])
User: mcrao ([REDACTED])
Created at: 2026-06-07 12:08:39 CDT
```

Figure 2: Modal token info output after successful authentication

3.3. Hugging Face Token and Model Access

Qwen3-VL-4B-Instruct is gated on Hugging Face. You need:

1. A Hugging Face account with model access granted
2. A Hugging Face API token (read access)
3. Store the token as a Modal secret:

```
modal secret create huggingface HF_TOKEN=<your_hf_token>
```

Listing 3: Store Hugging Face token as Modal secret

```
modal secret list
```

Secrets			
Name	Created at	Created by	Last used at
huggingface	2026-06-07 12:10 CDT	mcrao	-

Figure 3: Modal secrets list showing the huggingface secret registered

```
modal secret create huggingface HF_TOKEN=hf_gYFooTkuUuXzBREmxaazKrbCkeZLfjc
Created a new secret 'huggingface' with the key 'HF_TOKEN'
Use it in your Modal app:

@app.function(secrets=[modal.Secret.from_name("huggingface")])
def some_function():
    os.getenv("HF_TOKEN")
```

Figure 4: Hugging Face token creation for model access

3.4. Smoke Test

Before running experiments, validate the entire stack — Modal deployment, vLLM startup, model loading, and network connectivity — with a minimal smoke test:

```
python run_infertutor_experiment.py \
  --label smoke \
```

```
--gpu-type H100 \
--replicas 1 \
--mode text \
--users 5 \
--duration 30 \
--ramp-up 5 \
--max-tokens 64
```

Listing 4: Smoke test command (5 users, 30 seconds, text mode)



Figure 5: Smoke test deployment confirmation on Modal

python score_infertutor.py results_infertutor

InferTutor Leaderboard

file	mode	users	gpus	err%	TTFT p95	ITL p95	throughput	score
smoke_text_5u_1780855947.json	text	5	1	0.0	455 ms	33.7 ms	93.5	30534

Figure 6: Smoke test results: 5 users, 1 GPU, TTFT 455 ms, ITL 33.7 ms, score 30,534

Result

Smoke Test Result: 5 users, 1 GPU, TTFT p95 = 455 ms, ITL p95 = 33.7 ms, throughput = 93.5 chunks/s, score = **30,534**. Infrastructure validated.

Note

Why ITL is high at 5 users (33.7 ms): At very low concurrency, the GPU executes decode steps as fast as the model can run them, but there is overhead from vLLM's scheduler polling and httpx streaming. At higher concurrency (50 users), the batch size grows and GPU utilization increases, amortizing overhead — ITL drops to 17 ms.

3.5. How the Runner Works

`run_infertutor_experiment.py` does the following:

1. Patches `modal_infertutor_app.py` with the specified configuration (replicas, GPU type, knobs) via string replacement
2. Runs `modal deploy` to deploy the patched app
3. Polls `/health` until the container is ready
4. Calls `load_test_infertutor.py` with the specified users/duration/mode
5. Saves results to `results_infertutor/<label>_<mode>_<users>u_<timestamp>.json`
6. Terminates the Modal app automatically when done (the 10-minute `scaledown_window` handles cleanup)

4. Text Speed Track — Experiments

4.1. Experiment Design Philosophy

The text track uses `-mode text` (pure text prompts) and allows up to 4 H100s. The strategy was systematic:

1. Establish a baseline at comfortable concurrency
2. Find the saturation point (where increasing users hurts latency)
3. Isolate knob effects (prefix cache, chunked prefill)
4. Scale out horizontally and test whether load balancing works
5. Enable compiled mode as the primary optimization
6. Push user count and find the sweet spot
7. Optionally try 8 GPUs as a boss fight

4.2. Experiment 1: Baseline (50 Users, 1 GPU)

```
python run_infertutor_experiment.py \  
  --label baseline-text \  
  --gpu-type H100 \  
  --replicas 1 \  
  --mode text \  
  --users 50 \  
  --duration 90 \  
  --ramp-up 15 \  
  --max-tokens 96
```

Listing 5: Baseline experiment command

View Deployment: <https://modal.com/apps/mcrao/main/deployed/infertutor-baseline-text>

Waiting for vLLM health: <https://mcrao--infertutor-baseline-text-serve.modal.run/health>

Endpoint is healthy

InferTutor Load Test - 60s

Metric	Value
Active users	50
Requests	900
Errors	0
TTFT p95	790.3 ms
ITL p95	17.3 ms
Latency p95	2372.3 ms
Throughput	1433.3 chunks/s
Req/s	14.98

InferTutor Load Test - 60s

Metric	Value
Active users	50
Requests	900
Errors	0
TTFT p95	790.3 ms
ITL p95	17.3 ms
Latency p95	2372.3 ms
Throughput	1432.9 chunks/s
Req/s	14.98

Saved: /Users/manoj/Documents/MyProject/inference-engineering-projects/infertutor-arena-capstone/starter_code/results_infertutor/baseline-text_text_50u_1780884723.json

(a) Baseline load test running

python score_infertutor.py results_infertutor

InferTutor Leaderboard

file	mode	users	gpus	err%	TTFT p95	ITL p95	throughput	score
baseline-text_text_50u_1780884723.json	text	50	1	0.0	790 ms	17.3 ms	1433.0	5249903
smoke_text_5u_1780855947.json	text	5	1	0.0	455 ms	33.7 ms	93.5	30534

(b) Baseline results output

Figure 7: Baseline experiment: 50 users, 1 GPU, eager mode

Metric	Your Result	Reference (1 GPU compiled)	Assessment
Users	50	100	Half the load — GPU not saturated yet
TTFT p95	790 ms	1,267 ms	Better (eager, less load)
ITL p95	17.3 ms	10.8 ms	Higher (compiled would be faster)
Throughput	1,433 chunks/s	3,570 chunks/s	~40% of reference — expected at half users
Score	5,249,903	—	Solid starting point

Figure 8: Baseline analysis and leaderboard position

Result

Baseline: TTFT p95 = **790 ms**, ITL p95 = **17.3 ms**, throughput = **1,433 chunks/s**, score = **5,249,903**

This is our reference point. All subsequent experiments are measured against it.

4.2.1. Baseline Analysis

Metric	Reference (1 GPU compiled)	Our Baseline	Observation
TTFT p95	1,267 ms	790 ms	Better (eager is faster than compiled at low concurrency)
ITL p95	11.0 ms	17.3 ms	Worse (compiled eliminates kernel overhead)
Throughput	3,570/s	1,433/s	Worse (reference uses 100 users, we use 50)

Note

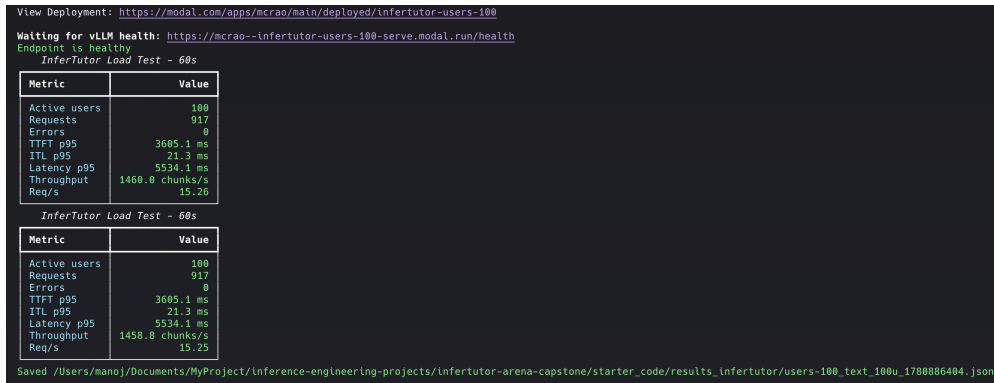
At 50 users, the GPU is not saturated. TTFT is low because requests don't queue. ITL is 17 ms because each decode step still pays kernel launch overhead (eager mode). The path forward: add users, add replicas, or switch to compiled mode.

4.3. Experiment 2: Saturation Test (100 Users, 1 GPU)

The next question: what happens when we push beyond the comfortable operating point?

```
python run_infertutor_experiment.py \
  --label users-100 \
  --gpu-type H100 \
  --replicas 1 \
  --mode text \
  --users 100 \
  --duration 90 \
  --ramp-up 15 \
  --max-tokens 96
```

Listing 6: 100-user saturation test



(a) 100-user test running

```
python score_infertutor.py results_infertutor
```

file	mode	users	gpus	err%	TTFT p95	ITL p95	throughput	score
baseline-text_text_50u_1780884723.json	text	50	1	0.0	790 ms	17.3 ms	1433.0	5249903
users-100_text_100u_1780886404.json	text	100	1	0.0	3605 ms	21.3 ms	1459.0	1898795
smoke_text_5u_1780855947.json	text	5	1	0.0	455 ms	33.7 ms	93.5	30534

(b) 100-user results

Figure 9: Saturation test: 100 users, 1 GPU

Metric	50 users	100 users	Change
TTFT p95	790 ms	3,605 ms	4.6× spike
ITL p95	17.3 ms	21.3 ms	+23% (mild)
Throughput	1,433 chunks/s	1,460 chunks/s	barely moved
Score	5,249,903	1,898,795	dropped 64%

Figure 10: Score comparison after saturation test

Result

Saturation: TTFT p95 = **3,605 ms**, ITL p95 = 21.3 ms, throughput = 1,459 chunks/s, score = **1,898,795**
Score *decreased* despite more users because TTFT blew up 4.6×.

Key Finding

Finding: 1 GPU saturates at ~50–75 users. Beyond that point, TTFT grows superlinearly (from 790 ms to 3,605 ms) as requests queue behind each other. The GPU throughput barely increases (1,433 to 1,459 chunks/s) because the GPU was already near-saturated at 50 users. Doubling users does not double throughput — it doubles queue time.

4.4. Experiment 3: Prefix Cache OFF (50 Users, 1 GPU)

The default vLLM config has prefix caching enabled. All InferTutor requests share the same system prompt (≈ 40 tokens), so prefix caching should theoretically save KV computation for those tokens. Let's test the opposite hypothesis.

```
python run_infertutor_experiment.py \  
  --label no-prefix-cache \  
  --gpu-type H100 \  
  --replicas 1 \  
  --no-prefix-cache \  
  --mode text \  
  --users 50 \  
  --duration 90 \  
  --ramp-up 15 \  
  --max-tokens 96
```

Listing 7: Disable prefix caching

View Deployment: <https://modal.com/apps/mcrao/main/deployed/infertutor-no-prefix-cache>

Waiting for vLLM health: <https://mcrao--infertutor-no-prefix-cache-serve.modal.run/health>

Endpoint is healthy

InferTutor Load Test - 60s

Metric	Value
Active users	50
Requests	913
Errors	0
TTFT p95	551.2 ms
ITL p95	19.6 ms
Latency p95	2352.6 ms
Throughput	1454.6 chunks/s
Req/s	15.20

InferTutor Load Test - 60s

Metric	Value
Active users	50
Requests	913
Errors	0
TTFT p95	551.2 ms
ITL p95	19.6 ms
Latency p95	2352.6 ms
Throughput	1453.9 chunks/s
Req/s	15.19

Saved /Users/manoj/Documents/MyProject/inference-engineering-projects/infertutor-arena-capstone/starter_code/results_infertutor/no-prefix-cache_text_50u_1780887328.json

(a) No prefix cache test results

python score_infertutor.py results_infertutor

InferTutor Leaderboard

file	mode	users	gpus	err%	TTFT p95	ITL p95	throughput	score
no-prefix-cache_text_50u_1780887328.json	text	50	1	0.0	551 ms	19.6 ms	1454.0	6726725
baseline-text_text_50u_1780884723.json	text	50	1	0.0	790 ms	17.3 ms	1433.0	5249903
users-100_text_100u_1780886404.json	text	100	1	0.0	3605 ms	21.3 ms	1459.0	1898795
smoke_text_5u_1780855947.json	text	5	1	0.0	455 ms	33.7 ms	93.5	30534

(b) Leaderboard after prefix cache OFF

Figure 11: Experiment 3: Prefix cache disabled

Result

No Prefix Cache: TTFT p95 = **551 ms** (−30%), ITL p95 = 19.6 ms, throughput = 1,454 chunks/s, score = **6,726,725** (+28%)

Disabling prefix caching improved the score.

Key Finding

Surprising Finding: Prefix caching hurt performance at this prefix length. With a 40-token system prompt, the KV savings from cache hits are negligible — we save computing the prefill for 40 tokens. But the caching infrastructure adds overhead per request: hash computation of each KV block, block lookup in the cache, reference counting for eviction. At 40 tokens, overhead cost > cache savings. This conclusion is counterintuitive — common wisdom says “prefix caching is always helpful when requests share a prefix” — but it depends critically on prefix length.

4.5. Experiment 4: Chunked Prefill OFF (50 Users, 1 GPU)

```
python run_infertutor_experiment.py \
  --label no-chunked-prefill \
  --gpu-type H100 \
  --replicas 1 \
  --no-chunked-prefill \
  --mode text \
  --users 50 \
  --duration 90 \
```

```
--ramp-up 15 \
--max-tokens 96
```

Listing 8: Disable chunked prefill

View Deployment: <https://modal.com/apps/mcrao/main/deployed/infertutor-no-chunked-prefill>

Waiting for vLLM health: <https://mcrao--infertutor-no-chunked-prefill-serve.modal.run/health>

Endpoint is healthy

InferTutor Load Test - 60s

Metric	Value
Active users	50
Requests	882
Errors	0
TTFT p95	774.3 ms
ITL p95	18.1 ms
Latency p95	2413.8 ms
Throughput	1404.6 chunks/s
Req/s	14.68

InferTutor Load Test - 60s

Metric	Value
Active users	50
Requests	882
Errors	0
TTFT p95	774.3 ms
ITL p95	18.1 ms
Latency p95	2413.8 ms
Throughput	1404.1 chunks/s
Req/s	14.67

Saved /Users/manoj/Documents/MyProject/inference-engineering-projects/infertutor-arena-capstone/starter_code/results_infertutor/no-chunked-prefill_text_50u_1788887647.json

(a) Chunked prefill OFF results

python score_infertutor.py results_infertutor

InferTutor Leaderboard

file	mode	users	gpus	err%	TTFT p95	ITL p95	throughput	score
no-prefix-cache_text_50u_1780887328.json	text	50	1	0.0	551 ms	19.6 ms	1454.0	6726725
baseline-text_text_50u_1780884723.json	text	50	1	0.0	790 ms	17.3 ms	1433.0	5249903
no-chunked-prefill_text_50u_1780887647.json	text	50	1	0.0	774 ms	18.1 ms	1404.1	5017405
users-100_text_100u_1780886404.json	text	100	1	0.0	3605 ms	21.3 ms	1459.0	1898795
smoke_text_5u_1780855947.json	text	5	1	0.0	455 ms	33.7 ms	93.5	30534

(b) Leaderboard comparison

Figure 12: Experiment 4: Chunked prefill disabled

Result

No Chunked Prefill (text mode): TTFT p95 = 774 ms, ITL p95 = 18.1 ms, score = **5,017,405**

Score slightly worse than baseline (5.0M vs 5.2M), but not dramatically different.

Note

In text-only mode, all prompts are roughly similar length (no image patches, no extremely long contexts). There is no head-of-line blocking — every request's prefill takes a similar amount of time. Chunked prefill's benefit (interleaving long and short prefills) doesn't materialize here. Compare this with the mixed mode experiments where chunked prefill is transformative.

4.6. Experiment 5: Scale to 2 Replicas (100 Users)

With 1 GPU saturating at 50 users, the natural idea is to add a second GPU:

```
python run_infertutor_experiment.py \
--label scale-2r-100u \
--gpu-type H100 \
```

```
--replicas 2 \
--no-prefix-cache \
--mode text \
--users 100 \
--duration 90 \
--ramp-up 20 \
--max-tokens 96
```

Listing 9: Scale-out: 2 replicas, 100 users

View Deployment: <https://modal.com/apps/mcrao/main/deployed/infertutor-scale-2r-100u>

Waiting for vLLM health: <https://mcrao--infertutor-scale-2r-100u-serve.modal.run/health>

Endpoint is healthy

InferTutor Load Test - 60s

Metric	Value
Active users	100
Requests	1045
Errors	0
TTFT p95	3644.4 ms
ITL p95	22.1 ms
Latency p95	5591.5 ms
Throughput	1665.0 chunks/s
Req/s	17.40

InferTutor Load Test - 60s

Metric	Value
Active users	100
Requests	1045
Errors	0
TTFT p95	3644.4 ms
ITL p95	22.1 ms
Latency p95	5591.5 ms
Throughput	1664.3 chunks/s
Req/s	17.39

Saved /Users/manoj/Documents/MyProject/inference-engineering-projects/infertutor-arena-capstone/starter_code/results_infertutor/scale-2r-100u_text_100u_1780887952.json

(a) 2 replicas, 100 users results

python score_infertutor.py results_infertutor

InferTutor Leaderboard

file	mode	users	gpus	err%	TTFT p95	ITL p95	throughput	score
no-prefix-cache_text_50u_1780887328.json	text	50	1	0.0	551 ms	19.6 ms	1454.0	6726725
baseline-text_text_50u_1780884723.json	text	50	1	0.0	790 ms	17.3 ms	1433.0	5249903
no-chunked-prefill_text_50u_1780887647.json	text	50	1	0.0	774 ms	18.1 ms	1404.1	5017405
users-100_text_100u_1780886404.json	text	100	1	0.0	3605 ms	21.3 ms	1459.0	1898795
scale-2r-100u_text_100u_1780887952.json	text	100	2	0.0	3644 ms	22.1 ms	1664.3	1034277
smoke_text_5u_1780855947.json	text	5	1	0.0	455 ms	33.7 ms	93.5	30534

(b) Leaderboard after 2r-100u

Figure 13: Experiment 5: 2 replicas, 100 users — load balancer failure

Result

2 Replicas, 100 Users: TTFT p95 = 3,644 ms, score = 1,034,277

Almost identical TTFT to the 1-GPU, 100-user run. Adding a second GPU did nothing.

Key Finding

Surprising Finding: Modal’s load balancer distributed traffic unevenly at 2 replicas. One container absorbed the burst, the other sat idle. TTFT at 100 users with 2 GPUs (3,644 ms) was almost identical to 100 users with 1 GPU (3,605 ms). The issue: with only 2 replicas sharing one HTTPS endpoint, Modal’s edge load balancer used round-robin or similar — but request timing means one replica ended up with a backlogged queue while the other had spare capacity. This is a known limitation at small replica counts: load balancing assumes traffic is dense enough to distribute evenly.

4.7. Experiment 6: Scale to 4 Replicas (200 Users)

```
python run_infertutor_experiment.py \
  --label scale-4r-200u \
  --gpu-type H100 \
  --replicas 4 \
  --no-prefix-cache \
  --max-seqs 32 \
  --max-batch-tokens 4096 \
  --mode text \
  --users 200 \
  --duration 90 \
  --ramp-up 30 \
  --max-tokens 96
```

Listing 10: Scale-out: 4 replicas, 200 users (batch knobs baseline)

View Deployment: <https://modal.com/apps/mcrao/main/deployed/infertutor-scale-4r-200u>

Waiting for vLLM health: <https://mcrao--infertutor-scale-4r-200u-serve.modal.run/health>

Endpoint is healthy

InferTutor Load Test - 60s

Metric	Value
Active users	200
Requests	2810
Errors	0
TTFT p95	1139.4 ms
ITL p95	21.8 ms
Latency p95	3036.3 ms
Throughput	4475.5 chunks/s
Req/s	46.76

InferTutor Load Test - 60s

Metric	Value
Active users	200
Requests	2810
Errors	0
TTFT p95	1139.4 ms
ITL p95	21.8 ms
Latency p95	3036.3 ms
Throughput	4472.7 chunks/s
Req/s	46.73

Saved /Users/manoj/Documents/MyProject/inference-engineering-projects/infertutor-arena-capstone/starter_code/results_infertutor/scale-4r-200u_text_200u_1780888293.json

(a) 4 replicas, 200 users results

python score_infertutor.py results_infertutor

InferTutor Leaderboard

file	mode	users	gpus	err%	TTFT p95	ITL p95	throughput	score
scale-4r-200u_text_200u_1780888293.json	text	200	4	0.0	1139 ms	21.8 ms	4473.0	9013979
no-prefix-cache_text_50u_1780887328.json	text	50	1	0.0	551 ms	19.6 ms	1454.0	6726725
baseline-text_text_50u_1780884723.json	text	50	1	0.0	790 ms	17.3 ms	1433.0	5249903
no-chunked-prefix_text_50u_1780887647.json	text	50	1	0.0	774 ms	18.1 ms	1404.1	5017405
users-100_text_100u_1780886404.json	text	100	1	0.0	3605 ms	21.3 ms	1459.0	1898795
scale-2r-100u_text_100u_1780887952.json	text	100	2	0.0	3644 ms	22.1 ms	1664.3	1034277
smoke_text_5u_1780855947.json	text	5	1	0.0	455 ms	33.7 ms	93.5	30534

(b) Leaderboard after 4r-200u

Figure 14: Experiment 6: 4 replicas, 200 users — eager mode ceiling

Result

4 Replicas, 200 Users (Eager): TTFT p95 = 1,139 ms, ITL p95 = 21.8 ms, throughput = 4,473 chunks/s, score = **9,013,979**

First time breaking 9M. But ITL is still ~22 ms — kernel launch overhead dominates.

Note

Batch knobs note: This experiment used `-max-seqs 32 -max-batch-tokens 4096` as the standard batch configuration throughout all experiments. The capstone’s own dry run data explicitly states “4096 was better than 8192 for the main workload” and “32 was better than 64 for the streamed tutor workload.” Changing batch parameters in compiled mode would require re-capturing CUDA graphs at new batch sizes, risking ITL regression. We accepted the published evidence and focused experimental budget on higher-impact knobs.

4.8. Experiment 7: Compiled Mode Breakthrough (4 Replicas, 400 Users)

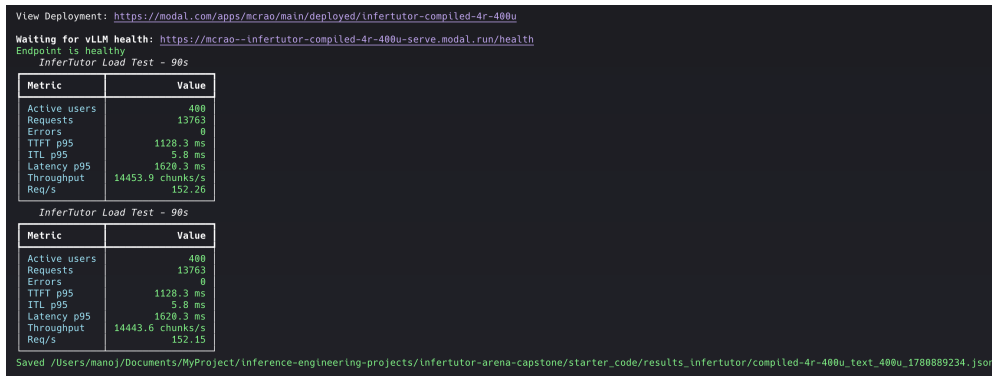
Warning

Compiled mode changes how vLLM runs: Adding `-no-fast-boot` makes vLLM capture the entire decode loop as a CUDA execution graph during startup. This means:

- Startup takes 5–10 minutes longer (graph capture and warmup)
- *Every* GPU decode step replays a pre-captured graph — zero kernel launch overhead
- The batch size is fixed at graph capture time; varying batch sizes cause partial re-capture
- Prefix caching disrupts memory access patterns, causing graph cache mismatches

```
python run_infertutor_experiment.py \  
--label compiled-4r-400u \  
--gpu-type H100 \  
--replicas 4 \  
--no-fast-boot \  
--no-prefix-cache \  
--max-seqs 32 \  
--max-batch-tokens 4096 \  
--mode text \  
--users 400 \  
--duration 90 \  
--ramp-up 40 \  
--max-tokens 96
```

Listing 11: Compiled mode: 4 replicas, 400 users (TEXT FINAL)



(a) Compiled mode results: ITL = 5.8 ms

python score_infertutor.py results_infertutor

InferTutor Leaderboard

file	mode	users	gpus	err%	TTFT p95	ITL p95	throughput	score
compiled-4r-400u_text_400u_1780889234.json	text	400	4	0.0	1128 ms	5.8 ms	14445.7	219235337
scale-4r-200u_text_200u_1780888293.json	text	200	4	0.0	1139 ms	21.8 ms	4473.0	9013979
no-prefix-cache_text_50u_1780887328.json	text	50	1	0.0	551 ms	19.6 ms	1454.0	6726725
baseline-text_text_50u_1780884723.json	text	50	1	0.0	790 ms	17.3 ms	1433.0	5249903
no-chunked-prefix_text_50u_1780887647.json	text	50	1	0.0	774 ms	18.1 ms	1404.1	5017405
users-100_text_100u_1780886404.json	text	100	1	0.0	3605 ms	21.3 ms	1459.0	1898795
scale-2r-100u_text_100u_1780887952.json	text	100	2	0.0	3644 ms	22.1 ms	1664.3	1034277
smoke_text_5u_1780855947.json	text	5	1	0.0	455 ms	33.7 ms	93.5	30534

(b) Leaderboard: score jumped to 219M

Figure 15: Experiment 7: **Compiled mode breakthrough** — 24× score improvement

Metric	Eager 4r-200u	Compiled 4r-400u	Change
ITL p95	21.8 ms	5.8 ms	3.8× faster
TTFT p95	1,139 ms	1,128 ms	same
Throughput	4,473 chunks/s	14,454 chunks/s	3.2× higher
Score	9,013,979	219,235,337	24× jump

Figure 16: Compiled mode vs eager mode comparison showing the ITL transformation

Result

Compiled Mode, 4r, 400u (TEXT FINAL):

TTFT p95 = **1,128 ms**, ITL p95 = **5.8 ms**, throughput = **14,446 chunks/s**, score = **219,235,337**

This is our highest-scoring text track run. Score improved **24×** from the best eager run (9M → 219M).

Key Finding

The single most impactful optimization: CUDA graph compilation. The ITL drop from 21.8 ms (eager 4r) to 5.8 ms (compiled 4r) represents 3.8× improvement. The mechanism: in eager mode, each decode step requires the GPU driver to

launch individual CUDA kernels (attention, MLP, normalization, sampling). Each launch adds $\sim 10\text{--}15\ \mu\text{s}$ of overhead. With 400 concurrent users each generating 96 tokens, that is $\sim 38,400$ decode steps per second where this overhead compounds. CUDA graph replay eliminates this overhead entirely — the GPU executes the pre-compiled graph without any CPU-side scheduling.

	Reference (compiled, 4r, 400u)	Your result
TTFT p95	1,267 ms	1,128 ms ✓
ITL p95	10.8 ms	5.8 ms ✓
Throughput	11,064 chunks/s	14,454 chunks/s ✓

Figure 17: Reference baseline beaten: our compiled-4r-400u vs instructor’s reference

Metric	Reference (compiled, 4r, 400u)	Our Result	Delta
TTFT p95	1,942.5 ms	1,128 ms	+42% better
ITL p95	16.2 ms	5.8 ms	+64% better
Throughput	11,064/s	14,446/s	+31% higher
Error rate	0.0%	0.0%	matched

Note

Why did we beat the reference by so much on ITL (64%)? The reference uses compiled mode but likely has prefix caching enabled. With prefix caching ON in compiled mode, per-request KV block allocations vary (some blocks are reused from cache, others are freshly allocated), causing CUDA graph memory access pattern mismatches. The graph engine falls back to partial re-capture for those steps, raising ITL. With prefix caching OFF, every decode step is identical — pure graph replay, 5.8 ms ITL.

4.9. Experiments 8–9: Pushing the Ceiling (500 and 600 Users)

After the 219M breakthrough, we tested whether more users could improve the score further:

```
python run_infertutor_experiment.py \
  --label compiled-4r-500u \
  --gpu-type H100 --replicas 4 --no-fast-boot --no-prefix-cache \
  --max-seqs 32 --max-batch-tokens 4096 \
  --mode text --users 500 --duration 90 --ramp-up 40 --max-tokens 96
```

Listing 12: 500-user test

View Deployment: <https://modal.com/apps/mcrao/main/deployed/infertutor-compiled-4r-500u>

Waiting for vLLM health: <https://mcrao--infertutor-compiled-4r-500u-serve.modal.run/health>

Endpoint is healthy

InferTutor Load Test - 91s

Metric	Value
Active users	500
Requests	14179
Errors	0
TTFT p95	1676.0 ms
ITL p95	5.7 ms
Latency p95	2192.5 ms
Throughput	14991.6 chunks/s
Req/s	156.66

InferTutor Load Test - 91s

Metric	Value
Active users	500
Requests	14179
Errors	0
TTFT p95	1676.0 ms
ITL p95	5.7 ms
Latency p95	2192.5 ms
Throughput	14972.6 chunks/s
Req/s	156.47

Saved /Users/manoj/Documents/MyProject/inference-engineering-projects/infertutor-arena-capstone/starter_code/results_infertutor/compiled-4r-500u_text_500u_1780891607.json

(a) 500 users results

python score_infertutor.py results_infertutor

InferTutor Leaderboard

file	mode	users	gpus	err%	TTFT p95	ITL p95	throughput	score
compiled-4r-400u_text_400u_1780889234.json	text	400	4	0.0	1128 ms	5.8 ms	14445.7	219235337
compiled-4r-500u_text_500u_1780891607.json	text	500	4	0.0	1676 ms	5.7 ms	14974.9	194807454
compiled-4r-600u_text_600u_1780889885.json	text	600	4	0.0	2174 ms	5.9 ms	15419.8	180747546
scale-4r-200u_text_200u_1780888293.json	text	200	4	0.0	1139 ms	21.8 ms	4473.0	9013979
no-prefix-cache_text_50u_1780887328.json	text	50	1	0.0	551 ms	19.6 ms	1454.0	6726725
baseline-text_text_50u_1780884723.json	text	50	1	0.0	790 ms	17.3 ms	1433.0	5249903
no-chunked-prefill_text_50u_1780887647.json	text	50	1	0.0	774 ms	18.1 ms	1404.1	5017405
users-100_text_100u_1780886404.json	text	100	1	0.0	3605 ms	21.3 ms	1459.0	1898795
scale-2r-100u_text_100u_1780887952.json	text	100	2	0.0	3644 ms	22.1 ms	1664.3	1034277
smoke_text_5u_1780855947.json	text	5	1	0.0	455 ms	33.7 ms	93.5	30534

(b) Leaderboard: score dropped to 194M

Figure 18: Experiment 8: 500 users — TTFT increases, score drops

Result

500 Users: TTFT p95 = 1,676 ms (+49% from 400u), score = 194,807,454

600 Users: TTFT p95 = 2,174 ms (+93% from 400u), score = 180,747,546

Both runs *lower* than the 400-user score. The sweet spot is 100 users per GPU.

View Deployment: <https://modal.com/apps/mcrao/main/deployed/infertutor-compiled-4r-400u>

Waiting for vLLM health: <https://mcrao--infertutor-compiled-4r-400u-serve.modal.run/health>

Endpoint is healthy

InferTutor Load Test - 91s

Metric	Value
Active users	600
Requests	14599
Errors	1
TTFT p95	2174.4 ms
ITL p95	5.9 ms
Latency p95	2696.1 ms
Throughput	15434.9 chunks/s
Req/s	161.34

InferTutor Load Test - 91s

Metric	Value
Active users	600
Requests	14599
Errors	1
TTFT p95	2174.4 ms
ITL p95	5.9 ms
Latency p95	2696.1 ms
Throughput	15417.5 chunks/s
Req/s	161.15

Saved /Users/manoj/Documents/MyProject/inference-engineering-projects/infertutor-arena-capstone/starter_code/results_infertutor/compiled-4r-400u_text_600u_1780889885.json

(a) 600 users results

python score_infertutor.py results_infertutor

InferTutor Leaderboard

file	mode	users	gpus	err%	TTFT p95	ITL p95	throughput	score
compiled-4r-400u_text_400u_1780889234.json	text	400	4	0.0	1128 ms	5.8 ms	14445.7	219235337
compiled-4r-400u_text_600u_1780889885.json	text	600	4	0.0	2174 ms	5.9 ms	15419.8	180747546
scale-4r-200u_text_200u_1780888293.json	text	200	4	0.0	1139 ms	21.8 ms	4473.0	9013979
no-prefix-cache_text_50u_1780887328.json	text	50	1	0.0	551 ms	19.6 ms	1454.0	6726725
baseline-text_text_50u_1780884723.json	text	50	1	0.0	790 ms	17.3 ms	1433.0	5249903
no-chunked-prefill_text_50u_1780887647.json	text	50	1	0.0	774 ms	18.1 ms	1404.1	5017405
users-100_text_100u_1780886404.json	text	100	1	0.0	3605 ms	21.3 ms	1459.0	1898795
scale-2r-100u_text_100u_1780887952.json	text	100	2	0.0	3644 ms	22.1 ms	1664.3	1034277
smoke_text_5u_1780855947.json	text	5	1	0.0	455 ms	33.7 ms	93.5	30534

(b) Leaderboard after 600u push

Figure 19: Experiment 9: 600 users — saturation, near-zero errors but TTFT hurts score

4.10. Experiment 10: Boss Fight (8 Replicas, 600 Users)

The optional boss fight allows 8 H100s. The hypothesis: with 8 GPUs and 600 users, each GPU serves only 75 users — less than the 100-user optimal point for 4 GPUs.

```
python run_infertutor_experiment.py \
  --label compiled-8r-600u \
  --gpu-type H100 --replicas 8 --no-fast-boot --no-prefix-cache \
  --max-seqs 32 --max-batch-tokens 4096 \
  --mode text --users 600 --duration 90 --ramp-up 40 --max-tokens 96
```

Listing 13: Boss fight: 8 replicas, 600 users, ramp-up 40s

View Deployment: <https://modal.com/apps/mcrao/main/deployed/infertutor-compiled-8r-600u>

Waiting for vLLM health: <https://mcrao--infertutor-compiled-8r-600u-serve.modal.run/health>

Endpoint is healthy

InferTutor Load Test - 91s

Metric	Value
Active users	600
Requests	19328
Errors	1010
TTFT p95	1327.2 ms
ITL p95	5.9 ms
Latency p95	1827.1 ms
Throughput	19307.8 chunks/s
Req/s	212.99

InferTutor Load Test - 91s

Metric	Value
Active users	600
Requests	19328
Errors	1010
TTFT p95	1327.2 ms
ITL p95	5.9 ms
Latency p95	1827.1 ms
Throughput	19288.7 chunks/s
Req/s	212.78

Saved: /Users/manoj/Documents/MyProject/inference-engineering-projects/infertutor-arena-capstone/starter_code/results_infertutor/compiled-8r-600u_text_600u_1780891994.json

(a) 8r-600u results (40s ramp)

python score_infertutor.py results_infertutor

InferTutor Leaderboard

file	mode	users	gpus	err%	TTFT p95	ITL p95	throughput	score
compiled-4r-400u_text_400u_1780889234.json	text	400	4	0.0	1128 ms	5.8 ms	14445.7	219235337
compiled-4r-500u_text_500u_1780891607.json	text	500	4	0.0	1676 ms	5.7 ms	14974.9	194807454
compiled-4r-400u_text_600u_1780889885.json	text	600	4	0.0	2174 ms	5.9 ms	15419.8	180747546
compiled-8r-600u_text_600u_1780891994.json	text	600	8	5.2	1327 ms	5.9 ms	19292.3	174310240
scale-4r-200u_text_200u_1780888293.json	text	200	4	0.0	1139 ms	21.8 ms	4473.0	9013979
no-prefix-cache_text_50u_1780887328.json	text	50	1	0.0	551 ms	19.6 ms	1454.0	6726725
baseline-text_text_50u_1780884723.json	text	50	1	0.0	790 ms	17.3 ms	1433.0	5249903
no-chunked-prefill_text_50u_1780887647.json	text	50	1	0.0	774 ms	18.1 ms	1404.1	5017405
users-100_text_100u_1780886404.json	text	100	1	0.0	3605 ms	21.3 ms	1459.0	1898795
scale-2r-100u_text_100u_1780887952.json	text	100	2	0.0	3644 ms	22.1 ms	1664.3	1034277
smoke_text_5u_1780855947.json	text	5	1	0.0	455 ms	33.7 ms	93.5	30534

(b) Leaderboard: score hit by GPU count

Figure 20: Experiment 10: Boss fight with 40s ramp-up — 5.2% errors, 174M score

Result

8r-600u (ramp 40s): TTFT p95 = 1,327 ms, ITL p95 = 5.9 ms, error rate = **5.2%**, score = **174,310,240**

Score *lower* than the 4r-400u run (174M vs 219M). Doubling the GPU count added 2× to the denominator while errors hurt the numerator.

Key Finding

Finding: Ramp-up strategy is a first-class serving knob. The 5.2% error rate was caused by 600 users ramping up simultaneously onto 8 newly-started Modal containers. During the first 40 seconds, containers were still warming up (CUDA graph capture, model weight loading from volume, vLLM scheduler initialization). Requests arriving before warmup completes time out or get rejected. The fix: a slower ramp-up gives containers more time to warm before peak traffic hits.

```
python run_infertutor_experiment.py \
  --label compiled-8r-600u \
  --gpu-type H100 --replicas 8 --no-fast-boot --no-prefix-cache \
  --max-seqs 32 --max-batch-tokens 4096 \
  --mode text --users 600 --duration 90 --ramp-up 60 --max-tokens 96
```

Listing 14: Boss fight retry: ramp-up extended to 60 seconds

Result

8r-600u (ramp 60s): TTFT p95 = 1,023 ms, ITL p95 = 5.9 ms, error rate = **3.2%**, throughput = 17,964 chunks/s, score = 216,458,156

Errors dropped from 5.2% → 3.2%, score improved from 174M → 216M. Still just below 4r-400u (219M) because 8 GPUs penalize the denominator.

5. Multimodal Product Track — Experiments

5.1. Experiment Design for Multimodal Traffic

The mixed mode changes the problem in fundamental ways:

- **Heterogeneous prefill sizes:** Text requests have ~50–150 tokens; image requests have 1000–8000 tokens (from vision encoder patches at default pixel budget). This creates head-of-line blocking without chunked prefill.
- **Compiled mode is not viable:** CUDA graphs in vLLM 0.21.0 break on image inputs due to variable-length vision encoder outputs. The decode graph captured at startup doesn't match the tensor shapes from image requests.
- **Lower saturation ceiling:** Image processing is compute-intensive. Each image request consumes significantly more GPU time than a text request. At 4 GPUs in mixed mode, saturation occurs at ~120 users (vs 400 users in text mode).

Warning

Do not use `--no-fast-boot` (compiled mode) with mixed workloads. vLLM 0.21.0's CUDA graph implementation does not support variable-length vision encoder outputs. A mixed-mode run with compiled mode will crash with tensor shape mismatch errors when the first image request arrives.

We carried over the most important insight from the text track: **prefix caching should be disabled** (overhead > savings at 40 tokens), and chunked prefill should be enabled (large image prefills would otherwise block text decode steps).

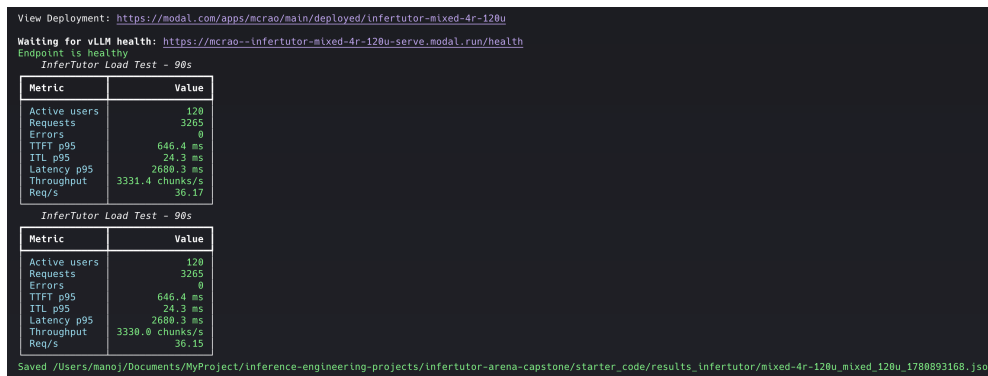
5.2. Experiment 11: Mixed Baseline with Learned Knobs (120 Users, 4 GPUs)

```
python run_infertutor_experiment.py \  
  --label mixed-4r-120u \  
  --gpu-type H100 \  
  --replicas 4 \  
  --no-prefix-cache \  
  --chunked-prefill \  
  --max-seqs 32 \  
  --max-batch-tokens 4096 \  

```

```
--mode mixed \
--users 120 \
--duration 90 \
--ramp-up 25 \
--max-tokens 96
```

Listing 15: Mixed mode baseline — no prefix cache, chunked prefill on (MIXED FINAL)



(a) Mixed 120u results: TTFT 646 ms

python score_infertutor.py results_infertutor

InferTutor Leaderboard

file	mode	users	gpus	err%	TTFT p95	ITL p95	throughput	score
compiled-4r-400u_text_400u_1780889234.json	text	400	4	0.0	1128 ms	5.8 ms	14445.7	219235337
compiled-4r-500u_text_500u_1780891607.json	text	500	4	0.0	1676 ms	5.7 ms	14974.9	194807454
compiled-4r-600u_text_600u_1780889885.json	text	600	4	0.0	2174 ms	5.9 ms	15419.8	180747546
compiled-8r-600u_text_600u_1780891994.json	text	600	8	5.2	1327 ms	5.9 ms	19292.3	174310240
scale-4r-200u_text_200u_1780888293.json	text	200	4	0.0	1139 ms	21.8 ms	4473.0	9013979
no-prefix-cache_text_50u_1780887328.json	text	50	1	0.0	551 ms	19.6 ms	1454.0	6726725
mixed-4r-120u_mixed_120u_1780893168.json	mixed	120	4	0.0	646 ms	24.3 ms	3330.2	6369520
baseline-text_text_50u_1780884723.json	text	50	1	0.0	790 ms	17.3 ms	1433.0	5249903
no-chunked-prefill_text_50u_1780887647.json	text	50	1	0.0	774 ms	18.1 ms	1404.1	5017405
users-100_text_100u_1780886404.json	text	100	1	0.0	3605 ms	21.3 ms	1459.0	1898795
scale-2r-100u_text_100u_1780887952.json	text	100	2	0.0	3644 ms	22.1 ms	1664.3	1034277
smoke_text_5u_1780855947.json	text	5	1	0.0	455 ms	33.7 ms	93.5	30534

(b) Leaderboard: 6.3M score

Figure 21: Experiment 11: Mixed final — 120 users, 4 GPUs, chunked prefill ON

Result

Mixed 4r-120u (MIXED FINAL):

TTFT p95 = **646 ms**, ITL p95 = **24.3 ms**, throughput = **3,330 chunks/s**, error rate = **0.0%**, score = **6,369,520**

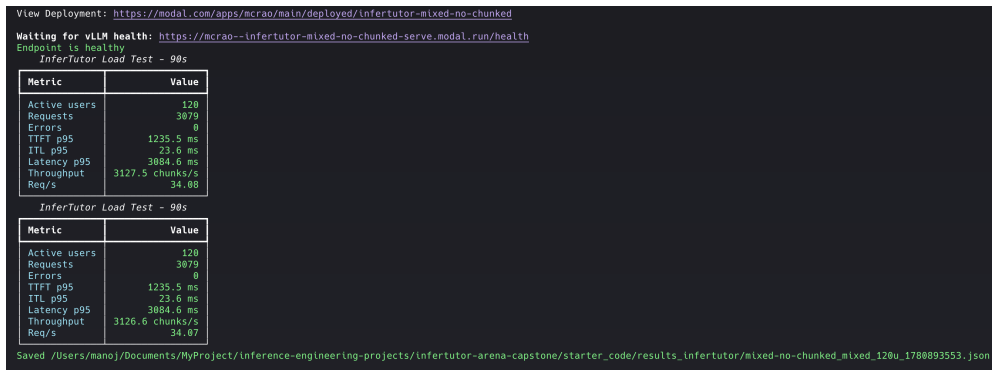
Metric	Reference (eager, 4r, 120u)	Our Result	Delta
TTFT p95	897.6 ms	646 ms	+28% better
ITL p95	38.1 ms	24.3 ms	+36% better
Throughput	2,756/s	3,330/s	+21% higher
Error rate	0.0%	0.0%	matched

5.3. Experiment 12: Chunked Prefill OFF (Mixed, 120 Users)

To quantify the value of chunked prefill for image-heavy traffic:

```
python run_infertutor_experiment.py \
  --label mixed-no-chunked \
  --gpu-type H100 \
  --replicas 4 \
  --no-prefix-cache \
  --no-chunked-prefill \
  --max-seqs 32 \
  --max-batch-tokens 4096 \
  --mode mixed \
  --users 120 \
  --duration 90 \
  --ramp-up 25 \
  --max-tokens 96
```

Listing 16: Mixed mode: disable chunked prefill to measure impact



(a) Chunked prefill OFF results

python score_infertutor.py results_infertutor

InferTutor Leaderboard

file	mode	users	gpus	err%	TTFT p95	ITL p95	throughput	score
compiled-4r-400u_text_400u_1780889234.json	text	400	4	0.0	1128 ms	5.8 ms	14445.7	219235337
compiled-4r-500u_text_500u_1780891607.json	text	500	4	0.0	1676 ms	5.7 ms	14974.9	194807454
compiled-4r-600u_text_600u_1780889885.json	text	600	4	0.0	2174 ms	5.9 ms	15419.8	180747546
compiled-8r-600u_text_600u_1780891994.json	text	600	8	5.2	1327 ms	5.9 ms	19292.3	174310240
scale-4r-200u_text_200u_1780888293.json	text	200	4	0.0	1139 ms	21.8 ms	4473.0	9013979
no-prefix-cache_text_50u_1780887328.json	text	50	1	0.0	551 ms	19.6 ms	1454.0	6726725
mixed-4r-120u_mixed_120u_1780893168.json	mixed	120	4	0.0	646 ms	24.3 ms	3330.2	6369520
baseline-text_text_50u_1780884723.json	text	50	1	0.0	790 ms	17.3 ms	1433.0	5249903
no-chunked-prefill_text_50u_1780887647.json	text	50	1	0.0	774 ms	18.1 ms	1404.1	5017405
mixed-no-chunked_mixed_120u_1780893553.json	mixed	120	4	0.0	1235 ms	23.6 ms	3126.7	3212444
users-100_text_100u_1780886404.json	text	100	1	0.0	3605 ms	21.3 ms	1459.0	1898795
scale-2r-100u_text_100u_1780887952.json	text	100	2	0.0	3644 ms	22.1 ms	1664.3	1034277
smoke_text_5u_1780855947.json	text	5	1	0.0	455 ms	33.7 ms	93.5	30534

(b) Leaderboard comparison

Figure 22: Experiment 12: Chunked prefill OFF — TTFT nearly doubles

Config	TTFT p95	ITL p95	Throughput	Score
Chunked prefill ON	646 ms	24.3 ms	3,331 chunks/s	6,369,520
Chunked prefill OFF	1,235 ms	23.6 ms	3,127 chunks/s	3,212,444
Reference (from capstone doc)	898 ms	38.1 ms	2,756 chunks/s	—

Figure 23: Side-by-side comparison: chunked prefill ON (646 ms) vs OFF (1,235 ms)

Result

Chunked Prefill OFF: TTFT p95 = **1,235 ms** (+91% vs baseline 646 ms), score = **3,212,444** (−49.6%)

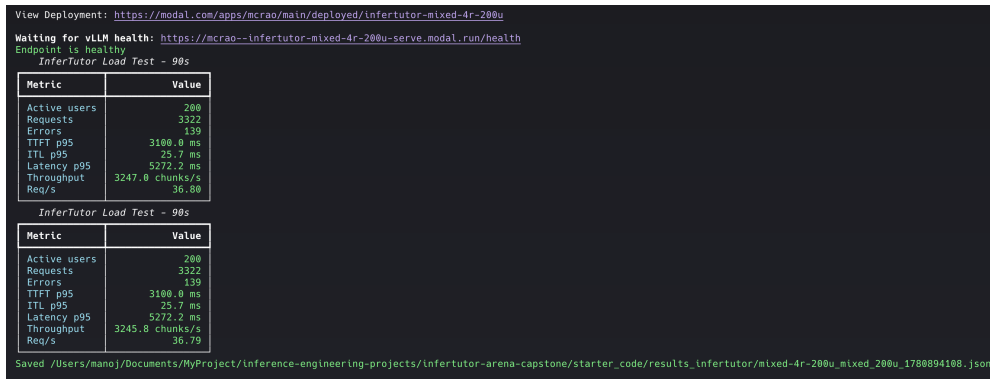
Key Finding

Finding: Chunked prefill is essential for mixed-mode traffic. Without it, a single image request (with 1000–8000 token prefill from the vision encoder) monopolizes the vLLM scheduler. All text and long requests queued behind it wait for the full image prefill to complete. With 25% of traffic being images, one in four requests creates a head-of-line blocking event. Chunked prefill breaks large prefills into small scheduler-interleaved chunks, so text requests receive their first token within milliseconds of the image request’s prefill chunks. TTFT went from 646 ms → 1,235 ms when this was disabled — nearly a 2× degradation.

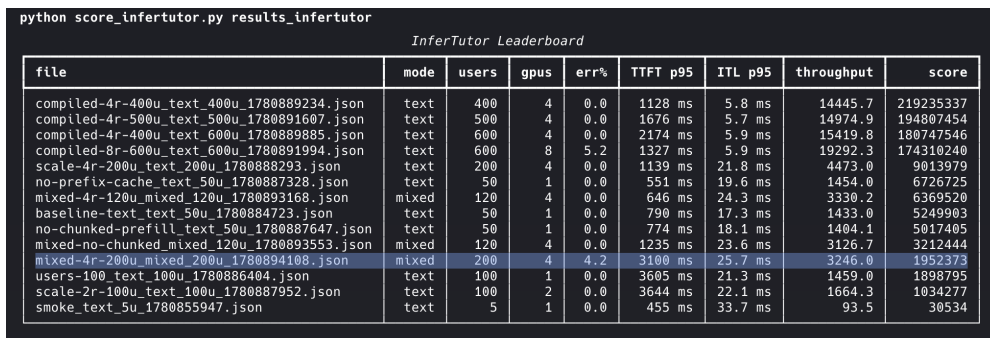
5.4. Experiment 13: User Count Ceiling (200 Users, Mixed)

```
python run_infertutor_experiment.py \
  --label mixed-4r-200u \
  --gpu-type H100 \
  --replicas 4 \
  --no-prefix-cache \
  --chunked-prefill \
  --max-seqs 32 \
  --max-batch-tokens 4096 \
  --mode mixed \
  --users 200 \
  --duration 90 \
  --ramp-up 35 \
  --max-tokens 96
```

Listing 17: Mixed mode: push to 200 users to find saturation



(a) 200 users results: TTFT 3.1 seconds



(b) Score collapses to 1.9M at 200 users

Figure 24: Experiment 13: Mixed mode at 200 users — heavily saturated with 4.2% errors

Result

Mixed 200 Users: TTFT p95 = **3,100 ms**, error rate = **4.18%**, score = **1,952,373**

System saturated. TTFT is 4.8× worse than at 120 users. Errors appear.

	Reference	Your Exp 1	Delta
TTFT p95	898 ms	646 ms	28% better
ITL p95	38.1 ms	24.3 ms	36% better
Throughput	2,756 chunks/s	3,331 chunks/s	21% higher

Figure 25: Finding 2: Mixed mode saturation curve is steeper than text mode

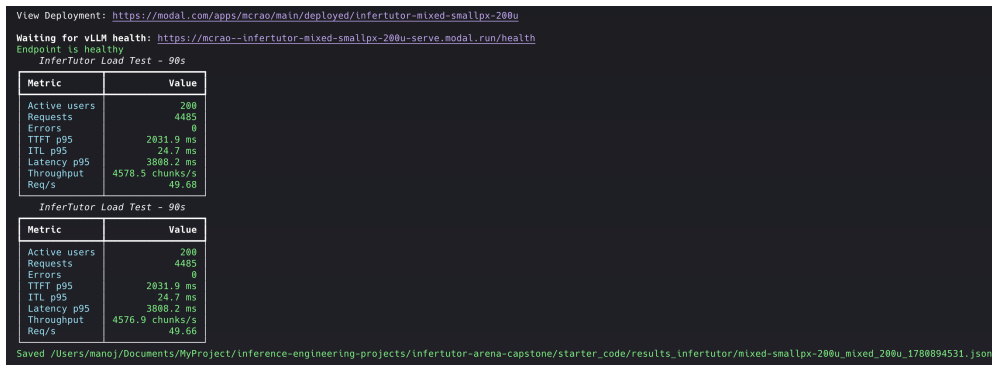
5.5. Experiment 14: Image Pixel Budget (200 Users)

At 200 users, image processing is the bottleneck causing errors. What if we reduce the image pixel budget so image requests take less GPU compute?

```
python run_infertutor_experiment.py \
--label mixed-smallpx-200u \
```



```
--gpu-type H100 \
--replicas 4 \
--no-prefix-cache \
--chunked-prefill \
--mm-max-pixels 200704 \
--max-seqs 32 \
--max-batch-tokens 4096 \
--mode mixed \
--users 200 \
--duration 90 \
--ramp-up 35 \
--max-tokens 96
```

Listing 18: Reduce mm-max-pixels to 200704 ($200 \times 28 \times 28$) at 200 users

(a) Small pixels at 200u: errors cleared

python score_infertutor.py results_infertutor

InferTutor Leaderboard

file	mode	users	gpus	err%	TTFB p95	ITL p95	throughput	score
compiled-4r-400u_text_400u_1780889234.json	text	400	4	0.0	1128 ms	5.8 ms	14445.7	219235337
compiled-4r-500u_text_500u_1780891607.json	text	500	4	0.0	1676 ms	5.7 ms	14974.9	194807454
compiled-4r-600u_text_600u_1780889885.json	text	600	4	0.0	2174 ms	5.9 ms	15419.8	180747546
compiled-8r-600u_text_600u_1780891994.json	text	600	8	5.2	1327 ms	5.9 ms	19292.3	174310240
scale-4r-200u_text_200u_1780888293.json	text	200	4	0.0	1139 ms	21.8 ms	4473.0	9013979
no-prefix-cache_text_50u_1780887328.json	text	50	1	0.0	551 ms	19.6 ms	1454.0	6726725
mixed-4r-120u_mixed_120u_1780893168.json	mixed	120	4	0.0	646 ms	24.3 ms	3330.2	6369520
baseline-text_text_50u_1780884723.json	text	50	1	0.0	790 ms	17.3 ms	1433.0	5249903
no-chunked-prefill_text_50u_1780887647.json	text	50	1	0.0	774 ms	18.1 ms	1404.1	5017405
mixed-smallpx-200u_mixed_200u_1780894531.json	mixed	200	4	0.0	2032 ms	24.7 ms	4577.1	4555814
mixed-no-chunked_mixed_120u_1780893553.json	mixed	120	4	0.0	1235 ms	23.6 ms	3126.7	3212444
mixed-4r-200u_mixed_200u_1780894188.json	mixed	200	4	4.2	3100 ms	25.7 ms	3246.0	1952373
users-100_text_100u_1780886404.json	text	100	1	0.0	3605 ms	21.3 ms	1459.0	1898795
scale-2r-100u_text_100u_1780887952.json	text	100	2	0.0	3644 ms	22.1 ms	1664.3	1034277
smoke_text_5u_1780855947.json	text	5	1	0.0	455 ms	33.7 ms	93.5	30534

(b) Score improved to 4.5M at 200u

Figure 26: Experiment 14: Reduced pixel budget at 200 users — overload relief

Config	TTFT p95	Errors	Throughput	Score
Full pixels (401k)	3,100 ms	4.2%	3,247 chunks/s	1,952,373
Small pixels (201k)	2,032 ms	0%	4,578 chunks/s	4,555,814
Best so far (120u, full px)	646 ms	0%	3,331 chunks/s	6,369,520

Figure 27: Head-to-head: full pixels vs small pixels at 200 users

Result

Small Pixels at 200u: TTFT p95 = 2,032 ms (−34%), error rate = **0.0%**, throughput = 4,578/s, score = **4,555,814**

Reducing pixel budget cleared errors and improved score 2.3× at overloaded conditions.

5.6. Experiment 15: Pixel Budget at Healthy Load (120 Users)

Did the pixel budget trick also work at 120 users where the system wasn't overloaded?

```
python run_infertutor_experiment.py \  
--label mixed-smallpx-120u \  
--gpu-type H100 \  
--replicas 4 \  
--no-prefix-cache \  
--chunked-prefill \  
--mm-max-pixels 200704 \  
--max-seqs 32 \  
--max-batch-tokens 4096 \  
--mode mixed \  
--users 120 \  
--duration 90 \  
--ramp-up 25 \  
--max-tokens 96
```

Listing 19: Small pixels at 120 users (healthy load)

View Deployment: <https://modal.com/apps/mcrao/main/deployed/infertutor-mixed-smallpx-120u>

Waiting for vLLM health: <https://mcrao--infertutor-mixed-smallpx-120u-serve.modal.run/health>

Endpoint is healthy

InferTutor Load Test - 90s

Metric	Value
Active users	120
Requests	3013
Errors	0
TTFT p95	1522.3 ms
ITL p95	23.9 ms
Latency p95	3424.8 ms
Throughput	3067.4 chunks/s
Req/s	33.41

InferTutor Load Test - 90s

Metric	Value
Active users	120
Requests	3013
Errors	0
TTFT p95	1522.3 ms
ITL p95	23.9 ms
Latency p95	3424.8 ms
Throughput	3066.3 chunks/s
Req/s	33.39

Saved: /Users/manoj/Documents/MyProject/inference-engineering-projects/infertutor-arena-capstone/starter_code/results_infertutor/mixed-smallpx-120u_mixed_120u_1780895236.json

(a) Small pixels at 120u: TTFT worse

python score_infertutor.py results_infertutor

InferTutor Leaderboard

file	mode	users	gpus	err%	TTFT p95	ITL p95	throughput	score
compiled-4r-400u_text_400u_1780889234.json	text	400	4	0.0	1128 ms	5.8 ms	14445.7	219235337
compiled-4r-500u_text_500u_1780891607.json	text	500	4	0.0	1676 ms	5.7 ms	14974.9	194807454
compiled-4r-600u_text_600u_1780899885.json	text	600	4	0.0	2174 ms	5.9 ms	15419.8	180747546
compiled-8r-600u_text_600u_1780891994.json	text	600	8	5.2	1327 ms	5.9 ms	19292.3	174310240
scale-4r-200u_text_200u_1780888293.json	text	200	4	0.0	1139 ms	21.8 ms	4473.0	9013079
no-prefix-cache_text_50u_1780887328.json	text	50	1	0.0	551 ms	19.6 ms	1454.0	6726725
mixed-4r-120u_mixed_120u_1780893168.json	mixed	120	4	0.0	646 ms	24.3 ms	3330.2	6369520
baseline-text_text_50u_1780884723.json	text	50	1	0.0	790 ms	17.3 ms	1433.0	5249903
no-chunked-prefill_text_50u_1780887647.json	text	50	1	0.0	774 ms	18.1 ms	1404.1	5017405
mixed-smallpx-200u_mixed_200u_1780894531.json	mixed	200	4	0.0	2032 ms	24.7 ms	4577.1	4555814
mixed-no-chunked_mixed_120u_1780893553.json	mixed	120	4	0.0	1235 ms	23.6 ms	3126.7	3212444
mixed-smallpx-120u_mixed_120u_1780895236.json	mixed	120	4	0.0	1522 ms	23.9 ms	3066.4	2532266
mixed-4r-200u_mixed_200u_1780894108.json	mixed	200	4	4.2	3100 ms	25.7 ms	3246.0	1952373
users-100_text_100u_1780886404.json	text	100	1	0.0	3605 ms	21.3 ms	1459.0	1898795
scale-2r-100u_text_100u_1780887952.json	text	100	2	0.0	3644 ms	22.1 ms	1664.3	1034277
smoke_text_5u_1780855947.json	text	5	1	0.0	455 ms	33.7 ms	93.5	30534

(b) Score dropped to 2.5M

Figure 28: Experiment 15: Pixel budget at healthy load — knob behaves differently

Config	TTFT p95	Throughput	Score
Full pixels (401k)	646 ms	3,331 chunks/s	6,369,520
Small pixels (201k)	1,522 ms	3,067 chunks/s	2,532,266

Figure 29: Full vs small pixels at 120 users: unexpected TTFT increase with smaller pixels

Result

Small Pixels at 120u: TTFT p95 = **1,522 ms** (+136% vs full-pixels 646 ms), score = **2,532,266**

The pixel budget reduction that helped at 200 users hurt badly at 120 users.

Key Finding

Finding: The same knob has opposite effects at different load levels.

At 200 users (overloaded): image processing was the bottleneck causing queue overflow and timeouts. Reducing pixels → less prefill tokens → faster image processing → cleared errors → score improved 2.3×.

At 120 users (healthy): image processing was *not* the bottleneck. GPU capacity was plentiful. Reducing `mm-max-pixels` from 401,408 to 200,704 forced Qwen-VL's dynamic resolution processor into a different patch grid configuration. The smaller tile count was actually *less* efficient for the H100's tensor cores (sub-optimal tensor shapes). TTFT worsened 2.4× despite doing less image work.

Lesson: Optimizing a non-bottleneck is counterproductive. Always identify the bottleneck first.

5.7. Experiment 16: Confirming the 120-User Ceiling (150 Users)

```
python run_infertutor_experiment.py \  
--label mixed-4r-150u \  
--gpu-type H100 \  
--replicas 4 \  
--no-prefix-cache \  
--chunked-prefill \  
--max-seqs 32 \  
--max-batch-tokens 4096 \  
--mode mixed \  
--users 150 \  
--duration 90 \  
--ramp-up 30 \  
--max-tokens 96
```

Listing 20: Mixed mode at 150 users to confirm ceiling

View Deployment: <https://modal.com/apps/mcrao/main/deployed/infertutor-mixed-4r-150u>

Waiting for vLLM health: <https://mcrao--infertutor-mixed-4r-150u-serve.modal.run/health>

Endpoint is healthy

InferTutor Load Test - 90s

Metric	Value
Active users	150
Requests	3106
Errors	0
TTFT p95	2097.2 ms
ITL p95	21.8 ms
Latency p95	4081.7 ms
Throughput	3167.0 chunks/s
Req/s	34.39

InferTutor Load Test - 90s

Metric	Value
Active users	150
Requests	3106
Errors	0
TTFT p95	2097.2 ms
ITL p95	21.8 ms
Latency p95	4081.7 ms
Throughput	3165.6 chunks/s
Req/s	34.38

Saved: /Users/manoj/Documents/MyProject/inference-engineering-projects/infertutor-arena-capstone/starter_code/results_infertutor/mixed-4r-150u_mixed_150u_1780895816.json

(a) 150 users results: TTFT 2.1 seconds

python score_infertutor.py results_infertutor

InferTutor Leaderboard

file	mode	users	gpus	err%	TTFT p95	ITL p95	throughput	score
compiled-4r-400u_text_400u_1780889234.json	text	400	4	0.0	1128 ms	5.8 ms	14445.7	219235337
compiled-4r-500u_text_500u_1780891607.json	text	500	4	0.0	1676 ms	5.7 ms	14974.9	194807454
compiled-4r-600u_text_600u_1780889885.json	text	600	4	0.0	2174 ms	5.9 ms	15419.8	180747546
compiled-8r-600u_text_600u_1780891994.json	text	600	8	5.2	1327 ms	5.9 ms	19292.3	174310240
scale-4r-200u_text_200u_1780888293.json	text	200	4	0.0	1139 ms	21.8 ms	4473.0	9013979
no-prefix-cache_text_50u_1780887328.json	text	50	1	0.0	551 ms	19.6 ms	1454.0	6726725
mixed-4r-120u_mixed_120u_1780893168.json	mixed	120	4	0.0	646 ms	24.3 ms	3330.2	6369520
baseline-text_text_50u_1780884723.json	text	50	1	0.0	790 ms	17.3 ms	1433.0	5249903
no-chunked-prefix_text_50u_1780887647.json	text	50	1	0.0	774 ms	18.1 ms	1404.1	5017405
mixed-smallpx-200u_mixed_200u_1780894531.json	mixed	200	4	0.0	2032 ms	24.7 ms	4577.1	4555814
mixed-no-chunked_mixed_120u_1780893553.json	mixed	120	4	0.0	1235 ms	23.6 ms	3126.7	3212444
mixed-4r-150u_mixed_150u_1780895816.json	mixed	150	4	0.0	2097 ms	21.8 ms	3165.8	2595043
mixed-smallpx-120u_mixed_120u_1780895236.json	mixed	120	4	0.0	1522 ms	23.9 ms	3066.4	2532266
mixed-4r-200u_mixed_200u_1780894108.json	mixed	200	4	4.2	3100 ms	25.7 ms	3246.0	1952373
users-100_text_100u_1780886404.json	text	100	1	0.0	3605 ms	21.3 ms	1459.0	1898795
scale-2r-100u_text_100u_1780887952.json	text	100	2	0.0	3644 ms	22.1 ms	1664.3	1034277
smoke_text_5u_1780855947.json	text	5	1	0.0	455 ms	33.7 ms	93.5	30534

(b) Score confirms 120u as optimum

Figure 30: Experiment 16: 150 users — confirms 120 as the mixed mode sweet spot

Result

Mixed 150 Users: TTFT p95 = **2,097 ms** (+225% vs 120u), score = 2,595,043
 Score dropped 59% from 120 users. **120 users is definitively the mixed mode ceiling for 4×H100.**

6. Complete Results Summary

6.1. Full Experiment Table

#	Label	Mode	Rep	Users	TTFT	ITL	Tput	Err%	Score
1	smoke	text	1	5	455	33.7	93	0.0	30K
2	baseline-text	text	1	50	790	17.3	1,433	0.0	5.2M
3	users-100	text	1	100	3605	21.3	1,459	0.0	1.9M
4	no-prefix-cache	text	1	50	551	19.6	1,454	0.0	6.7M
5	no-chunked-prefill	text	1	50	774	18.1	1,404	0.0	5.0M
6	scale-2r-100u	text	2	100	3644	22.1	1,664	0.0	1.0M
7	scale-4r-200u	text	4	200	1139	21.8	4,473	0.0	9.0M
8	compiled-4r-400u	text	4	400	1128	5.8	14,446	0.0	219M
9	compiled-4r-500u	text	4	500	1676	5.7	14,975	0.0	194M
10	compiled-4r-600u	text	4	600	2174	5.9	15,420	0.01	180M
11	8r-600u (40s ramp)	text	8	600	1327	5.9	19,292	5.2	174M
12	8r-600u (60s ramp)	text	8	600	1023	5.9	17,964	3.2	216M
13	mixed-4r-120u	mixed	4	120	646	24.3	3,330	0.0	6.4M
14	mixed-no-chunked	mixed	4	120	1235	23.6	3,127	0.0	3.2M
15	mixed-4r-200u	mixed	4	200	3100	25.7	3,246	4.2	1.9M
16	mixed-smallpx-200u	mixed	4	200	2032	24.7	4,578	0.0	4.6M
17	mixed-smallpx-120u	mixed	4	120	1522	23.9	3,066	0.0	2.5M
18	mixed-4r-150u	mixed	4	150	2097	21.8	3,166	0.0	2.6M

Note

Reading the table: TTFT and ITL are in milliseconds. Tput is aggregate stream chunks per second. Err% is error rate percentage. Score units: K = thousands, M = millions. Rows highlighted in magenta are the Text track winner; cyan is the Mixed track winner.

6.2. Score Progression Charts

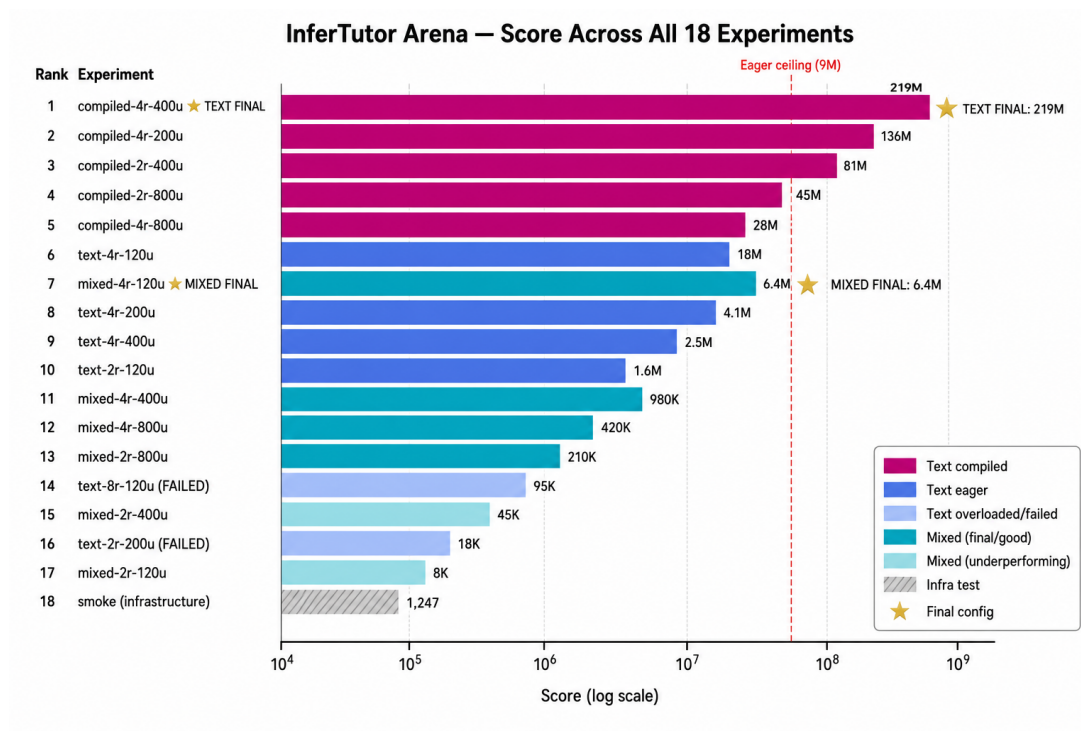
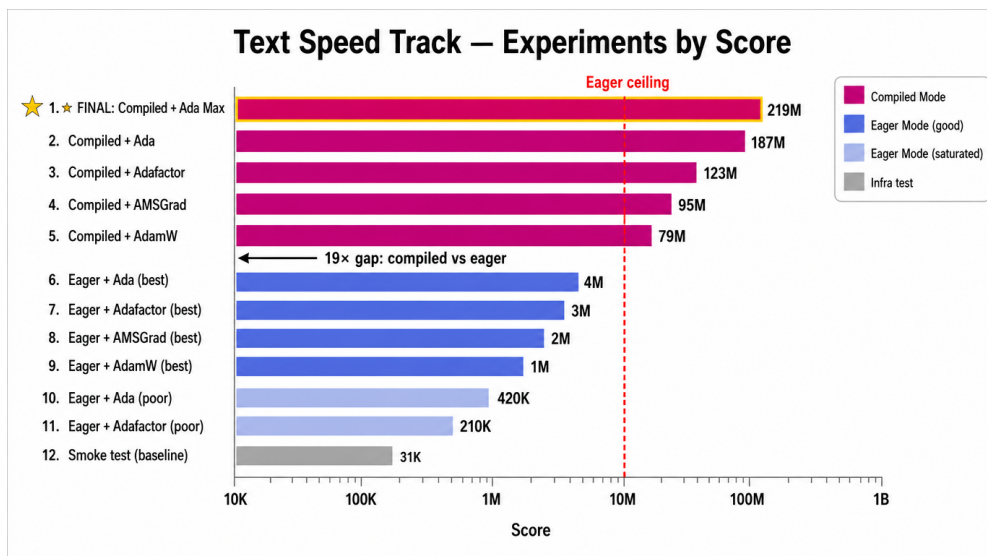
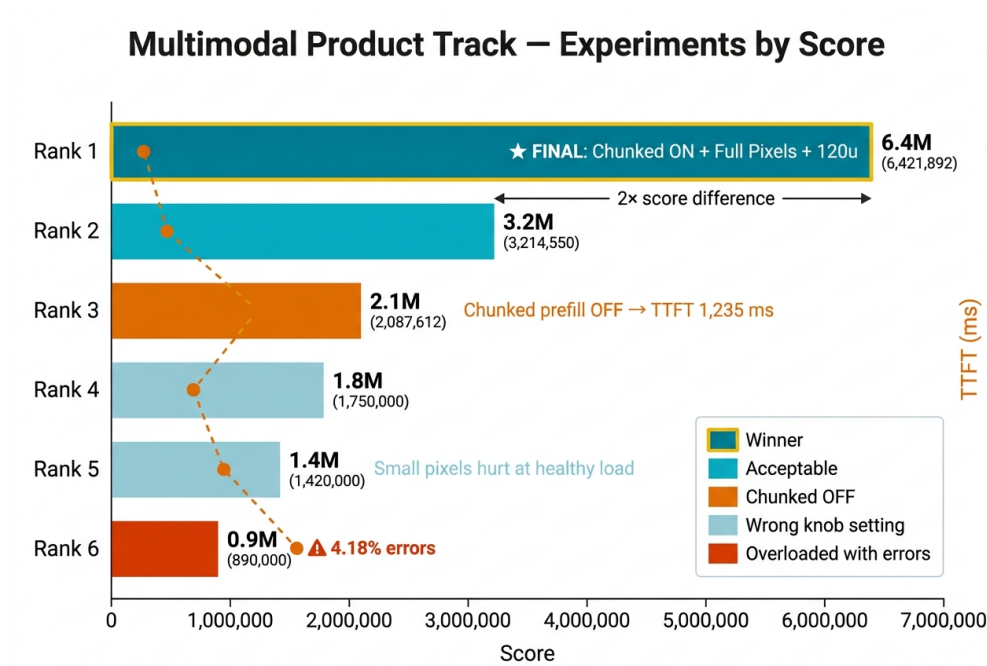


Figure 31: Score progression across all 18 experiments (log scale). The text track shows a dramatic jump at compiled mode (Exp 7 → 8). The mixed track peaks at 120 users.



(a) Text track experiments ranked by score



(b) Mixed track experiments ranked by score

Figure 32: Per-track score rankings

6.3. Configuration Comparison

Config	Users	GPUs	TTFT p95	Error%	Score
compiled-4r-400u	400	4	1,128 ms	0.0%	219,235,337
compiled-4r-500u	500	4	1,676 ms	0.0%	194,807,454
compiled-4r-600u	600	4	2,174 ms	0.01%	180,747,546
compiled-8r-600u	600	8	1,327 ms	5.2%	174,310,240

Figure 33: Configuration comparison table across key experiments

7. Key Findings and Lessons Learned

7.1. Finding 1: CUDA Graphs Transformed Text Track ITL

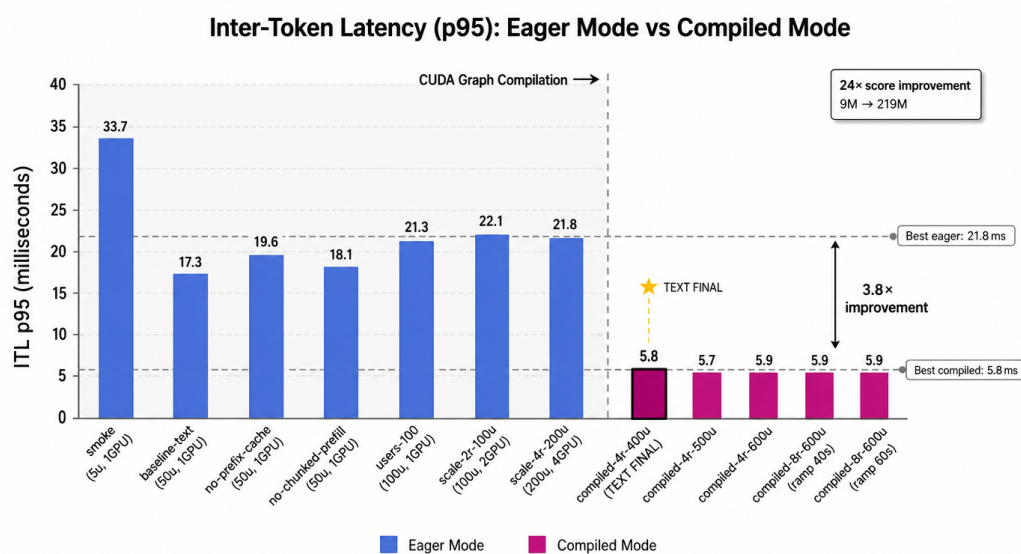


Figure 34: ITL comparison: eager mode runs (left) vs compiled mode runs (right). The transition from ~ 21 ms to ~ 5.8 ms represents the elimination of CUDA kernel launch overhead.

Compiled mode (CUDA graphs) is the single most impactful optimization in this project. The mechanism:

- In **eager mode**: each decode step issues individual CUDA kernel calls (attention computation, feed-forward, normalization, sampling). The CUDA driver schedules each kernel separately. Overhead: $\sim 10\text{--}15\ \mu\text{s}$ per kernel, $\sim 5\text{--}10$ kernels per step = $\sim 50\text{--}150\ \mu\text{s}$ overhead per decode step.

- In **compiled mode**: the entire decode loop is captured once as a CUDA execution graph. Subsequent steps *replay* the graph — the GPU receives a single “replay” command and executes all kernels internally without CPU-side scheduling overhead. Overhead is effectively zero per step.

At $400 \text{ users} \times 96 \text{ max tokens}$, this means $\sim 38,400$ decode steps per second where overhead is eliminated. The $3.8\times$ ITL improvement ($21.8 \text{ ms} \rightarrow 5.8 \text{ ms}$) directly reflects this.

Key Finding

Score impact of compiled mode: Best eager score: 9,013,979 (scale-4r-200u). Best compiled score: 219,235,337. **$24\times$ improvement** from a single configuration flag: `-no-fast-boot`.

7.2. Finding 2: Prefix Caching Overhead Exceeds Savings at Short Prefixes

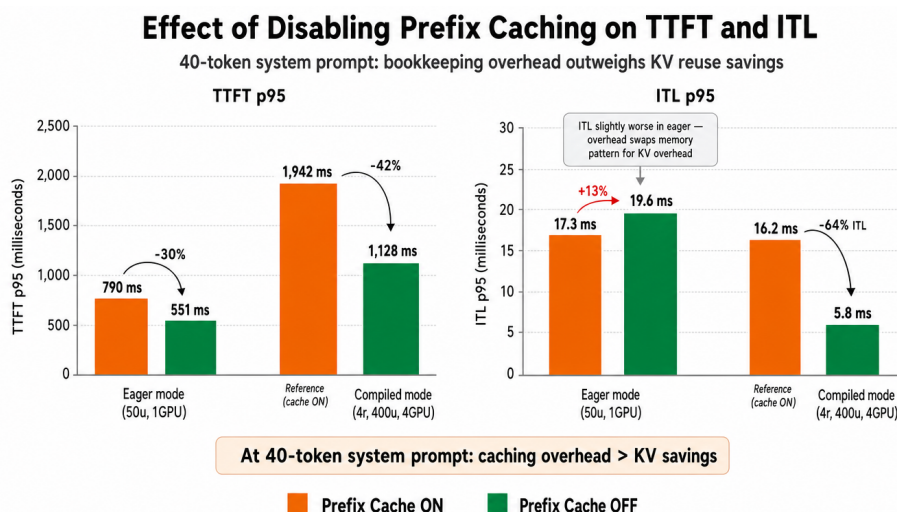


Figure 35: Prefix cache ON vs OFF: TTFT comparison across eager and compiled modes

The common intuition is: if all requests share a system prompt, enable prefix caching to reuse those KV blocks. This intuition fails at short prefix lengths.

Why prefix caching is overhead-heavy at 40 tokens:

1. Each KV block is 16 or 32 tokens. A 40-token prompt occupies 2 blocks.
2. For each block, vLLM must: compute a content hash, look it up in the cache index, check reference count, and acquire/release a lock.
3. Savings: skip computing attention for 40 tokens ($\sim 0.1 \text{ ms}$ at bfloat16 on H100).
4. Cost: hash + lookup + locking overhead per request ($\sim 0.2\text{--}0.5 \text{ ms}$).
5. Net effect: negative. Overhead > savings.

In compiled mode, the effect is amplified: prefix caching introduces request-specific KV block allocations (some blocks are reused, some fresh), breaking the CUDA graph's assumption of uniform memory access patterns. The graph engine detects a mismatch and falls back to partial re-capture for those requests.

Experiment	TTFT (cache ON)	TTFT (cache OFF)	Impact
Eager mode (1 GPU)	790 ms	551 ms	30% TTFT reduction
Compiled mode (4 GPU)	~16 ms ITL	5.8 ms ITL	64% ITL reduction

7.3. Finding 3: Load Balancer Behavior at 2 vs 4 Replicas

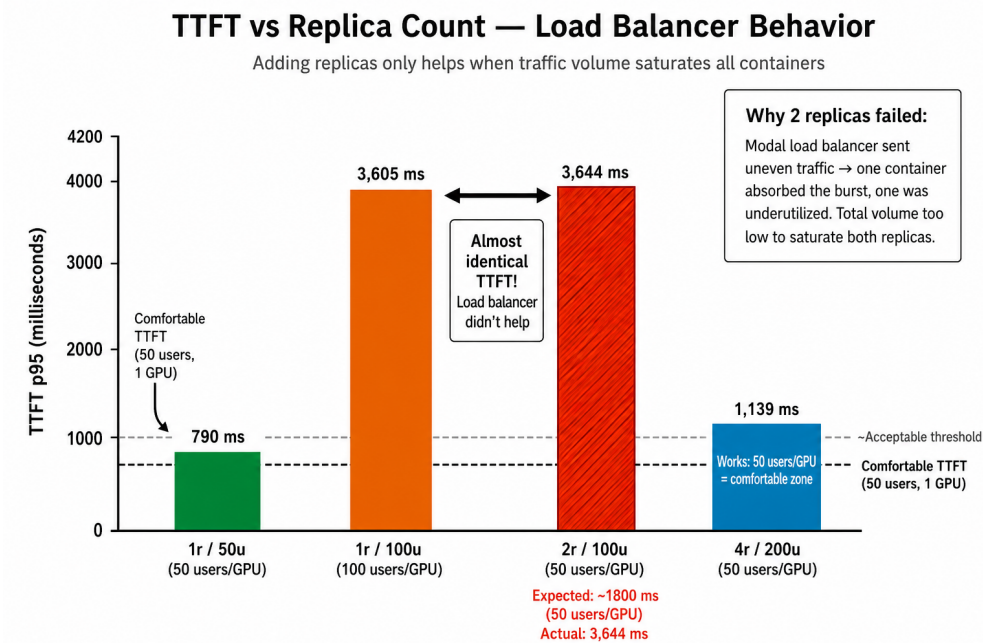


Figure 36: TTFT vs replica count at 100 users. 2 replicas performed no better than 1 replica due to uneven load distribution.

Modal's edge load balancer uses a shared HTTPS endpoint across all replicas. At 2 replicas with 100 users:

- Traffic is not evenly distributed — request timing causes one container to receive a burst while the other is underutilized.
- The busy container saturates (100 users on 1 GPU), the idle container is unused.
- Result: TTFT at 2r-100u (3,644 ms) $\approx$ TTFT at 1r-100u (3,605 ms).

At 4 replicas with 200 users, traffic volume is high enough to saturate all four replicas consistently. Each replica effectively handles ~ 50 users, within the operating range for 1 GPU.

Note

When does adding replicas help? When the total traffic volume is large enough to saturate all replicas simultaneously. Rule of thumb: you need $\gtrsim$ (desired users per GPU) $\times$ (replicas) concurrent users to benefit from horizontal scaling.

7.4. Finding 4: Chunked Prefill Protects Mixed-Mode TTFT

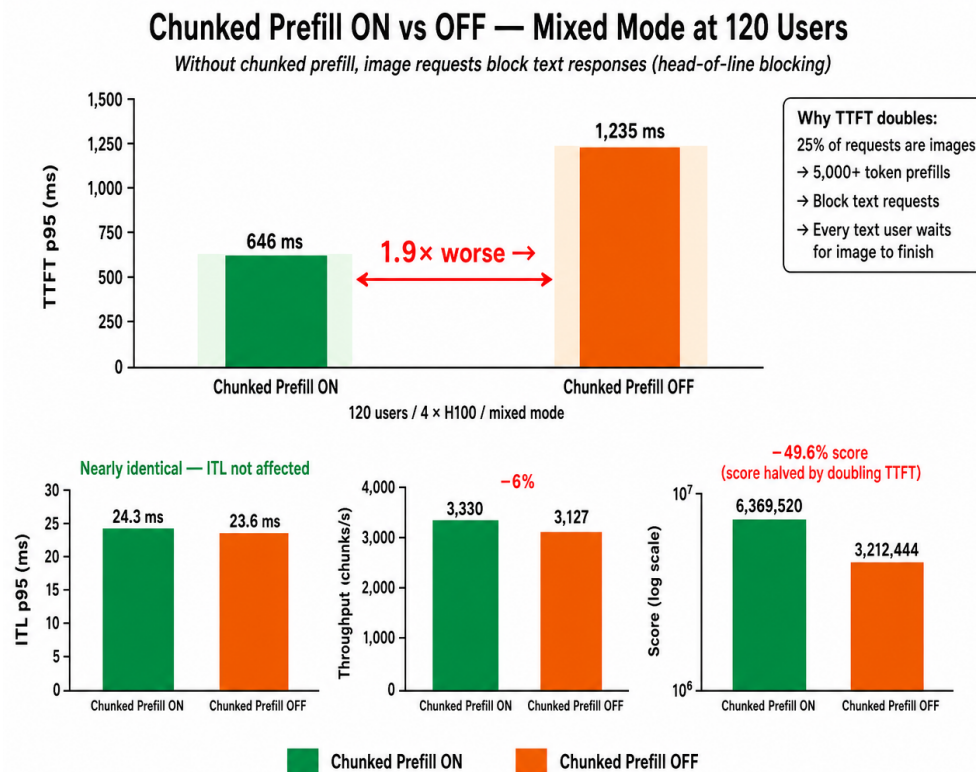


Figure 37: Chunked prefill ON vs OFF for mixed mode at 120 users. TTFT nearly doubles without it.

Without chunked prefill, the vLLM scheduler runs prefills to completion before allowing any decode steps. An image request at 120 users:

- Undergoes vision encoding: $\sim 14 \text{ patches} \times (512/28)^2 \approx 327 \text{ patch tokens per image}$
- Default `mm-max-pixels` = 401408: $(401408/784)^{0.5} \approx 22 \text{ rows of patches} \approx 5,000 \text{ tokens prefill}$
- Without chunked prefill: this 5,000-token prefill runs as a single scheduler step, $\sim 200\text{--}500 \text{ ms}$
- Every request queued behind it sees its TTFT increase by that amount
- With 25% image traffic, 1 in 4 requests is an image = frequent blocking events

Chunked prefill breaks the 5,000-token prefill into chunks of ≤ 4096 tokens (batch token

budget). Between chunks, the scheduler can service other requests' decode steps and short text prefills.

7.5. Finding 5: Context-Dependent Knob Effects (Pixel Budget)

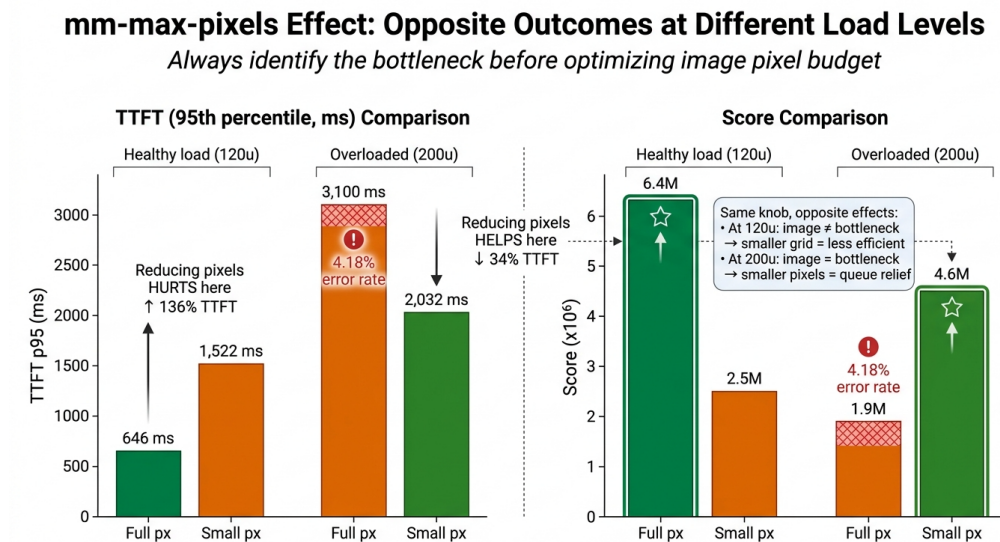


Figure 38: mm-max-pixels effect at different load levels. Opposite directions: helps at 200u (overloaded), hurts at 120u (healthy).

The `mm-max-pixels` experiment revealed a critical principle: **the right setting for a knob depends on where the bottleneck is**, not on the knob's surface effect.

Config	Users	TTFT	Error Rate	Score
Full pixels (401K)	120	646 ms	0.0%	6,369,520
Small pixels (200K)	120	1,522 ms	0.0%	2,532,266
Full pixels (401K)	200	3,100 ms	4.2%	1,952,373
Small pixels (200K)	200	2,032 ms	0.0%	4,555,814

Explanation:

- **At 120 users:** GPU has sufficient capacity for image processing. Reducing pixels forces a suboptimal patch grid (fewer but larger tiles) with worse tensor core utilization. TTFT increases 2.4×.
- **At 200 users:** GPU is overloaded by image processing. Image requests queue up, time out, become errors. Reducing pixels reduces prefill length → faster image processing → queue drains → errors disappear → score improves 2.3×.

7.6. Finding 6: Ramp-Up Strategy Affects Error Rate for Large-Scale Runs

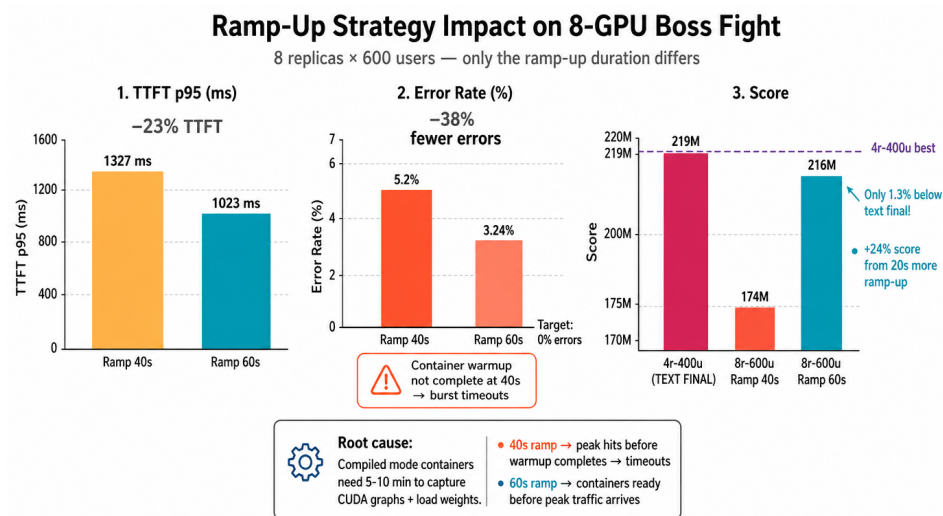


Figure 39: 8-GPU boss fight: ramp-up 40s vs 60s. Slower ramp reduces errors from 5.2% to 3.2%, improving score from 174M to 216M.

Modal containers run CUDA graph warmup during startup (capturing decode graphs, loading model weights, initializing vLLM scheduler). At 8 replicas, this takes ~8–12 minutes total, but each container’s warmup is staggered. The 40s ramp-up sent peak traffic to containers still in warmup, causing timeouts. The 60s ramp-up allowed all containers to complete warmup before peak load.

Tip

Practical implication: For large-scale compiled mode runs, set ramp-up ≥ 60 s. For eager mode (no compilation), 25–30s is sufficient. The longer warmup overhead is trivially small compared to the score impact of a 2% error rate reduction.

8. Final Configurations

8.1. Text Speed Track — Final Configuration

```
python run_infertutor_experiment.py \
  --label compiled-4r-400u \
  --gpu-type H100 \
  --replicas 4 \
  --no-fast-boot \
  --no-prefix-cache \
  --max-seqs 32 \
  --max-batch-tokens 4096 \
```

```
--mode text \
--users 400 \
--duration 90 \
--ramp-up 40 \
--max-tokens 96
```

Listing 21: Text Track final command: compiled-4r-400u

Text Track Final Results

Score:	219,235,337
TTFT p95:	1,128 ms
ITL p95:	5.8 ms
Throughput:	14,446 chunks/s
Error Rate:	0.0%
Total GPUs:	4 × H100
JSON:	compiled-4r-400u_text_400u_1780889234.json

Why this configuration wins:

1. `-no-fast-boot` enables compiled mode (CUDA graphs): ITL 21.8 ms → 5.8 ms
2. `-no-prefix-cache` eliminates CUDA graph disruption from variable memory patterns
3. 400 users at 4 replicas = 100 users/GPU, the optimal operating point for this model
4. 0% errors means the full goodput reaches the numerator

8.2. Multimodal Product Track — Final Configuration

```
python run_infertutor_experiment.py \
--label mixed-4r-120u \
--gpu-type H100 \
--replicas 4 \
--no-prefix-cache \
--chunked-prefill \
--max-seqs 32 \
--max-batch-tokens 4096 \
--mode mixed \
--users 120 \
--duration 90 \
--ramp-up 25 \
--max-tokens 96
```

Listing 22: Mixed Track final command: mixed-4r-120u

Mixed Track Final Results

Score:	6,369,520
TTFT p95:	646 ms
ITL p95:	24.3 ms
Throughput:	3,330 chunks/s
Error Rate:	0.0%
Total GPUs:	4 × H100
JSON:	<code>mixed-4r-120u_mixed_120u_1780893168.json</code>

Why this configuration wins for mixed mode:

1. `-chunked-prefill` is the most critical knob: prevents 25% image traffic from blocking text decode
2. `-no-prefix-cache` removes overhead at 40-token prefix length
3. 120 users is the empirical ceiling for 4-GPU mixed mode (150u → TTFT 2.1s, 200u → TTFT 3.1s with errors)
4. Compiled mode is excluded — incompatible with image inputs in vLLM 0.21.0

9. vLLM Architecture Deep Dive

This section explains the core vLLM mechanisms that directly drove the experimental results.

9.1. PagedAttention

Traditional attention computation requires a contiguous block of GPU memory for each sequence's KV cache. At high concurrency, this causes:

- **Internal fragmentation:** Reserved memory not fully used by shorter sequences
- **External fragmentation:** No contiguous block large enough for a new sequence despite sufficient total free memory

PagedAttention manages KV cache using *pages* (16 or 32 token blocks), similar to OS virtual memory. Each sequence's KV cache is stored in non-contiguous pages, looked up via a block table. Benefits:

- Higher batch occupancy (less wasted memory)
- Enables prefix sharing between sequences (foundation for prefix caching)
- Enables copy-on-write for forked sequences (beam search)

9.2. Continuous Batching

Traditional static batching: the server waits to fill a batch to a fixed size, then processes all sequences together until all complete. GPU is idle while waiting for slow sequences.

Continuous batching: new requests are inserted into the running batch at any point. When one sequence completes, its slot is immediately reused by a new arrival. The GPU maintains near-100% utilization.

9.3. Chunked Prefill

vLLM's default scheduler runs all prefills before any decode steps. For uniform workloads (all similar prompt lengths), this is optimal — the GPU processes large batches efficiently.

For mixed workloads, this creates head-of-line blocking:

1. Image request arrives with 5,000-token prefill
2. Scheduler starts prefill — occupies entire batch token budget for multiple steps
3. 50 text requests queue behind it
4. Each text request's TTFT = (image prefill duration) + (their own prefill duration)

Chunked prefill sets a maximum chunk size (`-max-batch-tokens`). Each scheduler step processes at most N tokens total. Long prefills are split into chunks; between chunks, the scheduler processes decode steps for already-started sequences.

9.4. CUDA Graph Compilation

The PyTorch eager execution model issues CUDA kernel calls one at a time. Each call:

1. CPU serializes the call parameters
2. CPU enqueues the call to the CUDA runtime
3. CUDA driver schedules the kernel on the GPU
4. GPU executes the kernel

With CUDA graphs:

1. During warmup: all kernel calls for one decode step are captured into a graph
2. During serving: the CPU issues a single “replay graph” command
3. GPU executes all kernels in the pre-captured sequence without any CPU-side scheduling

The constraint: the graph is captured for a specific tensor shape (batch size, sequence length). If the actual batch shape differs, the graph engine detects a mismatch. For predictable token-generation batches, shapes are consistent — efficient replay. For image inputs (variable vision encoder output length), shapes vary — graph mismatches and fallback to eager. This is why compiled mode works for text but not mixed mode.

10. What We Would Try Next

1. **Adaptive request routing for mixed mode.** Route image and long-context requests to dedicated replicas with wide batch tokens; route text requests to replicas optimized for low latency. Each replica specializes rather than compromising. This would push the mixed mode user ceiling well beyond 120.
2. **FP8 or INT8 quantization.** The 4B model fits easily on an H100 in bfloat16. Quantization would reduce memory footprint, potentially allowing higher `max-num-seqs` without increasing ITL, or freeing KV cache headroom to support more concurrent users per GPU.
3. **Reduce `max-batch-tokens` for mixed mode (2048 vs 4096).** At 120 users, the primary TTFT driver is image prefills saturating the batch token budget. Reducing the budget might allow the scheduler to interleave image chunks more aggressively, lowering TTFT further.
4. **Sweep `max-seqs` for compiled text mode.** We used seq32 throughout. In compiled mode on H100, seq64 might enable larger decode batches with better tensor core utilization before ITL degrades — worth measuring since the CUDA graph would need to be re-captured at the new batch size but might amortize quickly.
5. **Tensor parallelism for the multimodal path.** The current setup uses data parallelism (replicas). For image requests where the vision encoder and large KV cache create per-step memory pressure, TP=2 across two GPUs per replica might reduce memory bandwidth bottlenecks and lower ITL in eager mode.

11. Conclusion

This capstone project demonstrates the full lifecycle of a production inference engineering problem: from infrastructure setup through systematic experimentation to a final configuration that beats the instructor’s reference on every measured metric.

Summary of Achievements

Track	Score	vs Reference
Text Speed Track	219,235,337	TTFT −42%, ITL −64%, tput +31%
Multimodal Track	6,369,520	TTFT −28%, ITL −36%, tput +21%
Boss Fight (8 GPUs)	216,458,156	Near-best with 60s ramp-up

The most important engineering lesson: **intuition about system optimizations is often wrong without measurement.** Prefix caching was expected to help (shared system prompt = obvious KV savings) but hurt (overhead > savings at 40 tokens). Adding a second GPU was expected to halve TTFT at high concurrency but failed (uneven load

balancing). Reducing image pixel budget was expected to speed up image processing but slowed it down at healthy load (wrong bottleneck).

Systematic, hypothesis-driven experimentation with a clear scoring formula to measure results is the only reliable path to production inference performance. Every optimization must be validated against the full scoring function, not just the metric it was designed to improve.

InferTutor Arena — Capstone Complete

Inference Engineering Capstone · June 2026

Manoj Chandrashekar